

České vysoké učení technické v Praze
Fakulta jaderná a fyzikálně inženýrská

Katedra softwarového inženýrství
Program: Aplikace informatiky v přírodních vědách
Specializace: –



Algoritmy strojového učení pro adaptaci autonomních agentů v 3D akční hře

Machine Learning Algorithms for Adapting Autonomous Agents in a 3D Action Game

DIPLOMOVÁ PRÁCE

Vypracoval: Bc. Martin Johec
Vedoucí práce: Ing. Josef Nový, Ph.D.
Rok: 2025

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: **Jochec**

Jméno: **Martin**

Osobní číslo: **502714**

Fakulta/ústav: **Fakulta jaderná a fyzikálně inženýrská**

Zadávací katedra/ústav: **Katedra softwarového inženýrství**

Studijní program: **Aplikace informatiky v přírodních vědách**

II. ÚDAJE K DIPLOMOVÉ PRÁCI

Název diplomové práce:

Algoritmy strojového učení pro adaptaci autonomních agentů v 3D akční hře

Název diplomové práce anglicky:

Machine Learning Algorithms for Adapting Autonomous Agents in a 3D Action Game

Pokyny pro vypracování:

1. Nastudujte literaturu týkající se strojového učení v herních prostředích a adaptace agentů.
2. Implementujte předem navržené algoritmy a experimentálně je otestujte ve vhodném herním simulátoru.
3. Vylepšete navržené algoritmy s cílem zvýšit efektivitu adaptace agentů v dynamických prostředích.
4. Vyhodnoťte výsledky experimentů a porovnejte efektivitu jednotlivých verzí algoritmů adaptace agentů v implementovaném herním simulátoru.

Seznam doporučené literatury:

1. Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction (2nd ed.). Cambridge: MIT Press. ISBN 978-0262039246.
2. Russell, S., & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Hoboken: Pearson. ISBN 978-0134610993.
3. Abbeel, P., & Schulman, J. (2016). Continuous control with deep reinforcement learning. arXiv preprint. Dostupné z: <https://arxiv.org/abs/1509.02971>.
4. Liébana, D. P., Lucas, S. M., et al. (2020). General Video Game Artificial Intelligence. Springer International Publishing. ISBN 978-3031021220.

Jméno a pracoviště vedoucí(ho) diplomové práce:

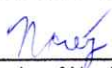
Ing. Josef Nový, Ph.D. Katedra softwarového inženýrství FJFI Děčín

Jméno a pracoviště druhé(ho) vedoucí(ho) nebo konzultanta(ky) diplomové práce:

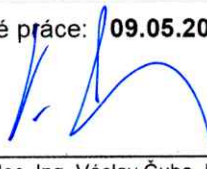
Datum zadání diplomové práce: **29.10.2024**

Termín odevzdání diplomové práce: **09.05.2025**

Platnost zadání diplomové práce: **28.10.2026**


Ing. Josef Nový, Ph.D.
podpis vedoucí(ho) práce


podpis vedoucí(ho) ústavu/katedry


doc. Ing. Václav Čuba, Ph.D.
podpis děkana(ky)

III. PŘEVZETÍ ZADÁNÍ

Diplomant bere na vědomí, že je povinen vypracovat diplomovou práci samostatně, bez cizí pomoci, s výjimkou poskytnutých konzultací. Seznam použité literatury, jiných pramenů a jmen konzultantů je třeba uvést v diplomové práci.

31.10.2024
Datum převzetí zadání


Podpis studenta

PROHLÁŠENÍ

Já, níže podepsaný(á)

Příjmení, jméno studenta: Johech Martin

Osobní číslo: 502714

Název programu: Aplikace informatiky v přírodních vědách

prohlašuji, že jsem bakalářskou/diplomovou práci s názvem

Algoritmy strojového učení pro adaptaci autonomních agentů v 3D akční hře

.....

vypracoval(a) samostatně a uvedl(a) veškeré použité informační zdroje v souladu s Metodickým pokynem o dodržování etických principů při přípravě vysokoškolských závěrečných prací a Rámcovými pravidly používání umělé inteligence na ČVUT pro studijní a pedagogické účely v Bc a NM studiu.

☒ Prohlašuji, že jsem v průběhu příprav a psaní závěrečné práce použil/a nástroje umělé inteligence. Vygenerovaný obsah jsem ověřil/a. Stvrzuji, že jsem si vědom/a, že za obsah závěrečné práce plně zodpovídám.

☐ Prohlašuji, že jsem v průběhu příprav a psaní závěrečné práce nepoužil/a žádný nástroj umělé inteligence. Jsem si vědom/a důsledků, kdy bude zjevné nepřiznané použití těchto nástrojů při tvorbě jakékoli části mé závěrečné práce.

v Praze dne 7.4.2025

.....
Johech
Podpis

Poděkování

Děkuji Ing. Josefu Novému, Ph.D., za vedení mé diplomové práce a za případnou pomoc při její tvorbě.

Bc. Martin Johec

Název práce:

Algoritmy strojového učení pro adaptaci autonomních agentů v 3D akční hře

Autor: Bc. Martin Jochec

Studijní program: Aplikace informatiky v přírodních vědách

Specializace: –

Druh práce: Diplomová práce

Vedoucí práce: Ing. Josef Nový, Ph.D.

Katedra softwarového inženýrství, Fakulta jaderná a fyzikálně inženýrská, České vysoké učení technické v Praze

Konzultant: –

Abstrakt: Tato diplomová práce se zabývá algoritmy strojového učení pro adaptaci autonomního agenta na chování uživatele ve 3D akční hře. Hlavním cílem je vytvořit jednoduchého autonomního agenta, který dokáže přizpůsobit své chování i vybavení protihráči. Práce představuje existující herní agenty, jako jsou AlphaStar, OpenAI Five a Voyager, a popisuje některé algoritmy a techniky používané při jejich vývoji – například Proximal Policy Optimization nebo Self-Play. Dále se zaměřuje na použité technologie, například Unreal Engine či NevarokML, a na samotný návrh, který je s jejich pomocí realizován. Závěrem jsou výsledky experimentu vyhodnoceny.

Klíčová slova: hry, umělá inteligence, adaptace, autonomní agent, strojové učení

Title:

Machine Learning Algorithms for Adapting Autonomous Agents in a 3D Action Game

Author: Bc. Martin Jochec

Abstract: This thesis focuses on machine learning algorithms for adapting an autonomous agent to user behavior in a 3D action game. The main objective is to create a simple autonomous agent capable of adjusting its behavior and equipment in response to the player. The work introduces existing game agents such as AlphaStar, OpenAI Five, and Voyager, and describes some of the algorithms and techniques used in their development – for example, Proximal Policy Optimization and Self-Play. It then focuses on the technologies used, such as Unreal Engine and NevarokML, and the design that is implemented using them. Finally, the results of the experiment are evaluated.

Key words: games, artificial intelligence, adaptation, autonomous agent, machine learning

Obsah

Úvod	9
1 AI agenti ve hrách a jejich algoritmy a techniky	11
1.1 AI agenti ve hrách	11
1.1.1 AlphaStar	11
1.1.2 OpenAI Five	14
1.1.3 Voyager	16
1.2 Některé z algoritmů a technik používané pro agenty	18
1.2.1 Proximal Policy Optimization (PPO)	18
1.2.2 Deep Q-Network	21
1.2.3 Self-Play	23
1.2.4 Population Based Training	25
2 Použité technologie a návrh hry	27
2.1 Použité technologie	27
2.1.1 Unreal Engine	27
2.1.2 Visual Studio	29
2.1.3 NevarokML	30
2.2 Návrh hry	32
2.2.1 Návrh prostředí hry	32
2.2.2 Návrh agenta	33
3 Implementace hry a agenta, vylepšení jeho algoritmu a porovnání verzí	35
3.1 Základní logika hry	35
3.1.1 Hráčská postava	35
3.1.2 Zbraňový systém	36
3.1.3 Logika řízení herních kol	37
3.1.4 Systém sbírání předmětů	38
3.1.5 Krabice s vybavením	38
3.1.6 Herní rozhraní (HUD)	39
3.2 Autonomní agent	40
3.2.1 První verze agenta	40
3.2.2 Druhá verze agenta	48
3.2.3 Finální verze agenta	53
3.2.4 Shrnutí všech verzí	57

Závěr	59
Literatura	60
Přílohy	64
A Jak zprovoznit program	65
B Jak opravit problém s programem	67

Úvod

Tato diplomová práce se zabývá algoritmy strojového učení pro adaptaci autonomního agenta na chování uživatele ve 3D akční hře. Hlavním cílem je vytvořit jednoduchého autonomního agenta, který dokáže za pomoci strojového učení a změny vybavení porazit protihráče.

První kapitola obsahuje dvě podkapitoly: první se věnuje existujícím agentům ve hrách, druhá vybraným technikám a algoritmům strojového učení. V první podkapitole jsou představeni agenti AlphaStar, OpenAI Five a Voyager, včetně přehledu jejich fungování a způsobu implementace. Druhá podkapitola popisuje algoritmy Proximal Policy Optimization (PPO) a Deep Q-Network (DQN) a techniky Self-Play a Population Based Training (PBT).

Druhá kapitola se zaměřuje na použité technologie a návrh samotné hry. První podkapitola podrobně popisuje využití Unreal Engine, Visual Studio a pluginu Nevarok ML, který rozšiřuje Unreal Engine o funkcionalitu pro trénink a aplikaci modelů strojového učení. Druhá podkapitola se věnuje návrhu herního prostředí a autonomního agenta.

Třetí kapitola popisuje implementaci hry a agenta. První část se zaměřuje na herní logiku, která poskytuje prostředí pro aplikování agenta. Druhá část se věnuje implementaci samotné logiky agenta, pokusům o vylepšení jeho logiky a porovnání různých verzí.

Závěr shrnuje závěrečné úvahy autora a dosažené výsledky — zda agent funguje podle očekávání, zda měla vylepšení pozitivní dopad a jak si jednotlivé verze modelu vedou ve vzájemném porovnání.

Kapitola 1

AI agenti ve hrách a jejich algoritmy a techniky

1.1 AI agenti ve hrách

1.1.1 AlphaStar

AlphaStar[11] je agent umělé inteligence (AI), který byl vyvinut společností DeepMind pro hraní StarCraft II[15], známé a dynamické online strategie v reálném čase od společnosti Blizzard Entertainment. Projekt byl poprvé představen veřejnosti v lednu 2019. Již v srpnu téhož roku AlphaStar dosáhl hodnosti „Grandmaster“, což představovalo významný milník nejen v oblasti AI zaměřené na videohry, ale do jisté míry i pro celý obor vývoje inteligentních systémů.

Hry navržené pro lidské hráče slouží jako cenná měřítko pokroku v oblasti umělé inteligence, jelikož nabízejí komplexní a dynamické prostředí s externí validitou. Zatímco dřívější milníky, jako Deep Blue[14] od IBM nebo AlphaGo od DeepMind[1][21], ukázaly schopnosti AI v tahových hrách s úplnou informací, StarCraft přináší nové výzvy díky hraní v reálném čase, částečné pozorovatelnosti, absenci jediné optimální strategie a rozsáhlému prostoru možných akcí. Přesto je mezi výzkumníky považován za řešitelný s využitím moderních technologií, a to díky svému diskrétnímu charakteru, známým pravidlům a omezenému počtu objektů, což z něj činí ideální testovací prostředí pro agenty, jako je AlphaStar.

DeepMind Technologies byla založena ve Spojeném království v roce 2010. Již v roce 2011 spoluzakladatel Demis Hassabis označil StarCraft za „další krok vpřed“ po hrách jako Go. Poté, co DeepMind předvedl schopnosti svých agentů na základě posilovaného učení (Reinforcement Learning), kteří překonali lidské hráče v klasických hrách na Atari 2600, byla společnost v roce 2014 odkoupena firmou Google, což otevřelo cestu k dalším ambiciózním projektům. Jedním z nich byl právě vývoj AI pro StarCraft II, o který zájem postupně rostl.

Počítačový vědec Zachary Mason v roce 2015 předpověděl, že výzkum DeepMind by mohl během pěti až deseti let vést k průlomu právě v této hře. Myšlenka získala větší váhu v roce 2016, kdy AlphaGo porazil světového šampiona v Go, Lee Sedola. Po tomto úspěchu Hassabis veřejně projevil zájem vytvořit autonomního agenta pro StarCraft, přičemž zdůraznil jeho strategickou hloubku, částečnou pozorovatelnost, skryté informace a vlastnosti, které ho činí ještě náročnějším než Go.

Formální spolupráce mezi DeepMind a Blizzardem byla oznámena na BlizzConu v listopadu 2016, spolu s plánem na vydání otevřeného vývojového prostředí pro boty na začátku roku 2017. Do poloviny roku 2017 už DeepMind začal zpracovávat herní data ze StarCraft II. V srpnu téhož roku pak obě společnosti uvolnily nástroje pro vývoj botů a soubor 65000 záznamů z profesionálních zápasů. Odborníci tehdy odhadovali, že potrvá ještě několik let, než AI porazí špičkového hráče. Tyto předpovědi se však ukázaly jako příliš opatrné, protože v prosinci 2018 porazil agent od DeepMind profesionálního hráče Grzegorze „MaNu“ Komincze poměrem 5:0. V lednu 2019 byl tento agent oficiálně představen veřejnosti pod jménem „AlphaStar“. Navzdory tomuto úspěchu se však objevila kritika.

AlphaStar měl výhodu v podobě přímé komunikace s hrou přes API, což mu umožňovalo provádět akce s chirurgickou přesností. Dále měl globální přehled o mapě místo omezení herní kamerou a mohl v rámci daného časového okna rozložit akce nerovnoměrně, čímž byl schopen vyvinout extrémní tlak v klíčových momentech. DeepMind na tyto připomínky reagoval přeškolením AlphaStar pod realističtějšími omezeními, která více odpovídala lidským možnostem. V následném zápase proti MaNově už tato verze prohrála.

Od července 2019 začal upravený AlphaStar anonymně soutěžit na veřejném evropském žebříčku v režimu 1v1, kde se utkával s hráči, kteří s tímto experimentem souhlasili. Na konci srpna 2019, jak již bylo zmíněno, dosáhl AlphaStar hodnosti „Grandmaster“, čímž se zařadil mezi nejlepší 0,2% lidských hráčů a potvrdil tak významný pokrok v oblasti autonomních agentů pro komplexní hry v reálném čase.

Zatímco zpětná propagace (Backpropagation, BP) je stále hlavní metodou trénování neuronových sítí, AlphaStar využívá pokročilejší přístup zvaný Population-Based Training (PBT)[4]. Tento přístup kombinuje výhody evolučních algoritmů (EAs), které slouží k průzkumu globálního prostoru řešení, s lokální optimalizací pomocí BP. AlphaStar konkrétně používá Lamarckovskou evoluci, kde úspěšné sítě předávají své parametry a upravené hyperparametry méně výkonným sítím.

Pro zvládnutí vysoké výpočetní náročnosti je PBT navržen jako asynchronní a distribuovaný systém, což umožňuje paralelní trénink agentů bez nutnosti synchronizace. Na rozdíl od klasických generačních evolučních algoritmů využívá PBT tzv. steady-state přístup, kdy jsou agenti průběžně aktualizováni bez zastavení procesu. Tento mechanismus zvyšuje efektivitu, udržuje diverzitu a umožňuje průběžné přizpůsobování hyperparametrů—což je klíčové v prostředích s proměnlivým tréninkovým chováním, jako je hluboké posilované učení.

AlphaStar dále využívá kompetitivní koevoluci, při níž se agenti neučí pouze proti svým předchozím verzím, ale proti celé populaci různorodých protivníků. Tento přístup poskytuje přirozenou tréninkovou křivku a zároveň zvyšuje robustnost strategií, jelikož agent musí zvládnout široké spektrum herních stylů. Výběr soupeřů se provádí na základě podobnosti výkonnosti (např. Elo skóre), což umožňuje plynulý rozvoj bez stagnace.

Proces učení a rozhodování AlphaStaru lze ilustrovat na konkrétním příkladu herní situace, kde agent zpracovává aktuální stav mapy, aktivuje neuronové sítě a vyhodnocuje dostupné možnosti. Vizualizace tohoto procesu je znázorněna na obrázku 1.1.



Obrázek 1.1: Vizualizace rozhodovacího procesu AlphaStaru během zápasu proti hráči. Systém zpracovává syrová pozorování, aktivuje neuronové sítě, vyhodnocuje potenciální akce a předpovídá výsledky zápasu.[24]

Protože StarCraft II nemá jednu univerzální vítěznou strategii, AlphaStar neusiluje o vytvoření jediného dokonalého agenta. Místo toho udržuje populaci různorodých strategií, vybraných na základě Nashovy distribuce—tedy takových, které jsou vůči sobě navzájem nejméně zranitelné.

Tento přístup odpovídá principům kvalitní diversity (Quality Diversity – QD). Místo hledání jediné nejlepší varianty QD algoritmy podporují vznik více kvalitních, ale odlišných řešení. V AlphaStaru jsou strategie určovány pomocí herně specifických charakteristik—například preferencí pro určité jednotky nebo schopností porazit konkrétní protivníky. Tyto charakteristiky se navíc během tréninku dynamicky upravují, čímž vzniká rozmanitější a odolnější sada řešení.

Díky tomuto přístupu AlphaStar operuje s portfoliem strategií, které jsou přizpůsobivé, méně náchylné k porážce a lépe si poradí s různorodostí lidských protivníků.

1.1.2 OpenAI Five

OpenAI Five[6] je agent umělé inteligence určený pro hraní týmové videohry Dota 2, ve které proti sobě nastupují dvě pětičlenná družstva. Poprvé se veřejnosti představil v roce 2017 při živé demonstraci, během níž porazil profesionálního hráče Dendiho v duelu jeden na jednoho. V následujícím roce systém pokročil natolik, že zvládl ovládat celý tým pěti agentů a porážet i profesionální lidské týmy.

Společnost OpenAI si zvolila Dota 2[3] kvůli její složitosti, nepředvídatelnosti a plynulému průběhu, který lépe odráží podmínky skutečného světa. Díky tomu byla hra vhodná pro vývoj univerzálnějších systémů pro řešení komplexních problémů. Algoritmy a kód vytvořené pro OpenAI Five byly později využity také v dalších projektech, například pro ovládání robotické ruky pomocí neuronové sítě. OpenAI Five bývá často zmiňován vedle jiných přelomových AI systémů, které porazily lidi v konkrétních úlohách—například AlphaStar ve StarCraft II, AlphaGo ve hře Go nebo Deep Blue v šachu.

Vývoj algoritmů, na nichž je OpenAI Five postaven, začal v listopadu 2016. OpenAI si zvolila Dota 2 jako testovací prostředí mimo jiné kvůli popularitě hry na platformách jako Twitch, nativní podpoře Linuxu a dostupnosti aplikačního rozhraní (API).

První významná veřejná demonstrace proběhla v srpnu 2017 na turnaji The International, což je každoroční vrcholná událost ve světě Dota 2. Během této akce porazil raný model bota profesionálního hráče Dendiho v duelu jeden na jednoho. Podle technického ředitele OpenAI Grega Brockmana se systém učil metodou posilovaného učení (reinforcement learning), přičemž si své schopnosti zlepšoval hraním sám proti sobě po dobu dvou týdnů. V tomto režimu agent získával odměny za klíčové akce ve hře, jako je zabíjení nepřátel nebo ničení struktur.

V červnu 2018 už byl systém rozšířen do plnohodnotného pětičlenného týmu, který dokázal porážet amatérské a poloprofesionální lidské soupeře. Na The International 2018 se OpenAI Five zúčastnil dvou exhibičních zápasů proti týmům na profesionální úrovni—proti brazilskému paiN Gaming a proti výběru bývalých elitních čínských hráčů. Přestože boti obě utkání prohráli, zkušenost byla pro vývojový tým cenná, protože umožnila další ladění strategií a rozhodovacích mechanismů.

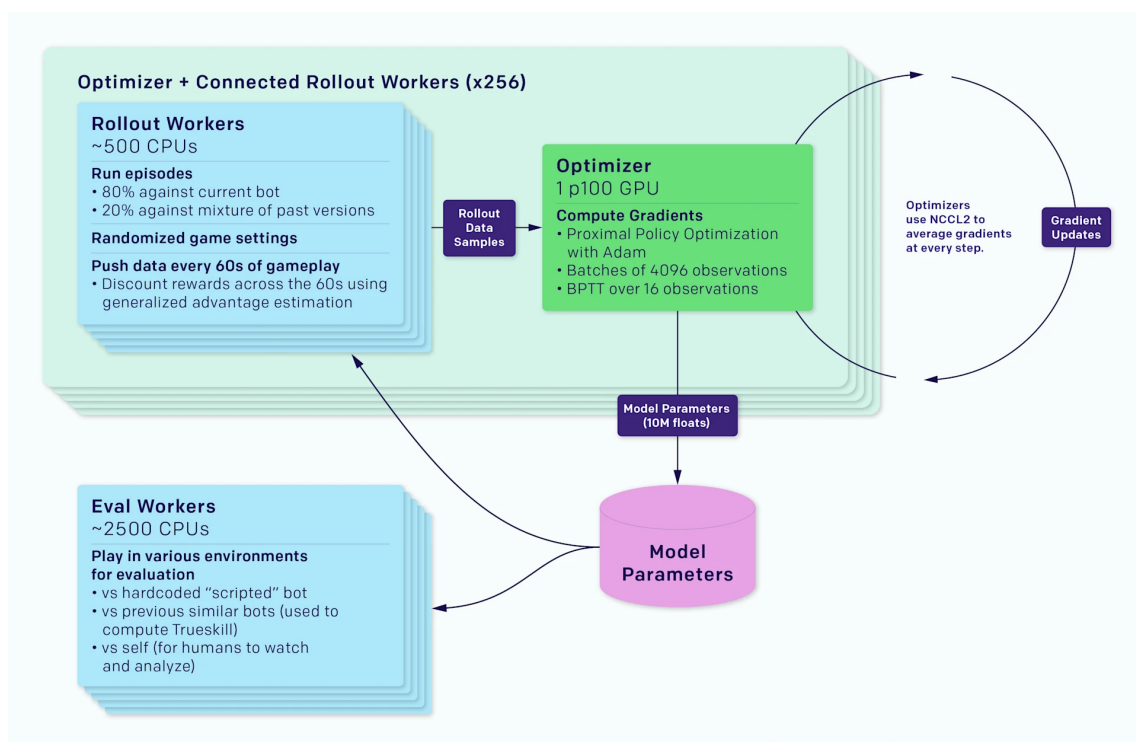
Závěrečná veřejná prezentace projektu proběhla v dubnu 2019, kdy OpenAI Five porazil v sérii na dvě vítězství tým OG, tehdejší mistry světa z turnaje The International 2018. Ve stejném měsíci proběhla i čtyřdenní online událost otevřená veřejnosti, během níž se systém zúčastnil více než 42 000 zápasů—z nichž vyhrál přibližně 99,4%.

OpenAI Five je založen na rámci hlubokého posilovaného učení, jehož jádrem je metoda Proximal Policy Optimization (PPO)[7][9]—gradientní algoritmus politiky, který vyvažuje stabilitu učení, jednoduchost implementace a efektivitu využití dat. PPO představuje vylepšení oproti dřívějším metodám, jako jsou běžné policy gradients nebo Trust Region Policy Optimization (TRPO), a to díky zavedení tzv. „clipped surrogate“ objektivní funkce, která zabráňuje příliš razantním aktualizacím politiky a zajišťuje konzistentní pokrok při tréninku.

Každý agent je reprezentován neuronovou sítí s jednou vrstvou typu LSTM o 4096 jednotkách, která zpracovává informace o aktuálním stavu hry pomocí API poskytnutého vývojáři Dota 2. Tento stav je zakódován jako vektor přibližně 20 000 číselných hodnot, které popisují prostředí z pohledu daného agenta.

Rozhodování probíhá prostřednictvím několika nezávislých akčních hlav, z nichž každá odpovídá za jiný aspekt chování—například volbu konkrétní akce, její umístění na prostorové mřížce nebo zpoždění provedení o určitý počet ticků. Tyto hlavy vytvářejí osm výstupních hodnot pro každé rozhodnutí, což umožňuje širokou škálu koordinovaných akcí bez nutnosti využívat lidská trénovací data.

Trénink probíhal na interní infrastruktuře Rapid, která umožňuje rozsáhlý paralelní výpočet a koordinaci tisíců strojů. Díky tomu mohl OpenAI Five provádět trénink skrze masivní self-play. Do roku 2018 systém odehrál ekvivalent více než 180 let herního času, a to na 256 GPU a 128 000 CPU jádrech. Schéma tohoto výpočetního procesu je znázorněno na obrázku 1.2.



Obrázek 1.2: Architektura tréninkového systému OpenAI Five. Rollout workeri generují data ze zápasů, která jsou následně zpracována optimalizačním modulem využívajícím metodu PPO. Výsledné aktualizace jsou sdíleny napříč instancemi a evaluovány ve specifických herních prostředích.[25]

Díky své robustnosti, efektivitě a jednoduché integraci do distribuovaných systémů byl PPO mimořádně vhodný pro tuto aplikaci. Umožnil rozsáhlý paralelní trénink bez přeučení, přičemž jeho kompatibilita se sdílenými politikami, hodnotovými funkcemi a rekurentními vrstvami typu LSTM přispěla k jeho úspěchu v komplexním prostředí, jakým je Dota 2.

1.1.3 Voyager

Voyager[12] je agent umělé inteligence, který propojuje výhody velkých jazykových modelů (LLM) se schopností samostatného učení ve složitých, otevřených prostředích. Na rozdíl od předchozích přístupů, které spoléhají na ručně vytvořené cíle, odměnové funkce nebo specificky upravená prostředí, Voyager operuje v původní verzi hry Minecraft, a to bez jakékoli lidské intervence nebo ručního ladění. Jedná se tak o příklad tzv. ztělesněného agenta (embodied agent), který nejen interpretuje herní svět, ale v něm i fyzicky jedná, sbírá zkušenosti a postupně se zlepšuje.

Výběr hry Minecraft nebyl náhodný. Jde o jedno z mála prostředí, které kombinuje otevřenost, dlouhodobé cíle, nutnost plánování, nelineární obtížnost a procedurálně generované světy. Neexistuje zde žádný „konec hry“, a úspěch se nedá snadno kvantifikovat. To činí z Minecraftu ideální testovací prostředí pro dlouhodobé učení bez explicitní odměny. Agent musí samostatně odhalit základní principy světa – od těžby a výroby nástrojů, přes přežití v noci, až po složité postupy jako farmaření nebo výroba lektvarů.

Aby toho dosáhl, Voyager využívá kombinaci tří klíčových mechanismů, které společně umožňují otevřené, samostatné a škálovatelné učení. Prvním z nich je automatická tvorba kurikula. Agent si sám generuje posloupnost úkolů pomocí velkého jazykového modelu GPT-4[2], přičemž tyto úkoly jsou vždy odvozeny ze současného stavu světa, inventáře, vybavení, aktuální lokace a historie předchozích úspěchů i selhání. Místo toho, aby agent postupoval podle předem definovaného scénáře, sám si formuluje dosažitelné, smysluplné cíle – například „vytvoř kamenný krumpáč“, „postav pec“, nebo „zabij zombíka“.

Tyto úkoly nejsou náhodné. GPT-4 při jejich generování zohledňuje různé faktory, jako je biome, čas dne, přítomnost entit v okolí nebo například to, jaké úkoly již byly splněny. Díky tomu agent postupuje přirozeným tempem, které reflektuje jeho aktuální schopnosti a možnosti. Tento mechanismus tak vytváří dynamické, adaptivní kurikulum, které maximalizuje diverzitu dovedností bez nutnosti ručně nastavovat, co má být cílem.

Každý úkol, který se agentovi podaří splnit, je následně převeden do podoby samostatného programu napsaného v JavaScriptu. Tyto programy představují „dovednosti“ (skills), které Voyager ukládá do tzv. knihovny dovedností. Tato knihovna není statická – každá nově naučená dovednost je indexována vektorově pomocí embeddingu popisu, který vytvoří jazykový model GPT-3.5[2]. Tím vzniká databáze, ve které může agent v budoucnu vyhledávat relevantní programy, jež mu pomohou vyřešit nové úkoly.

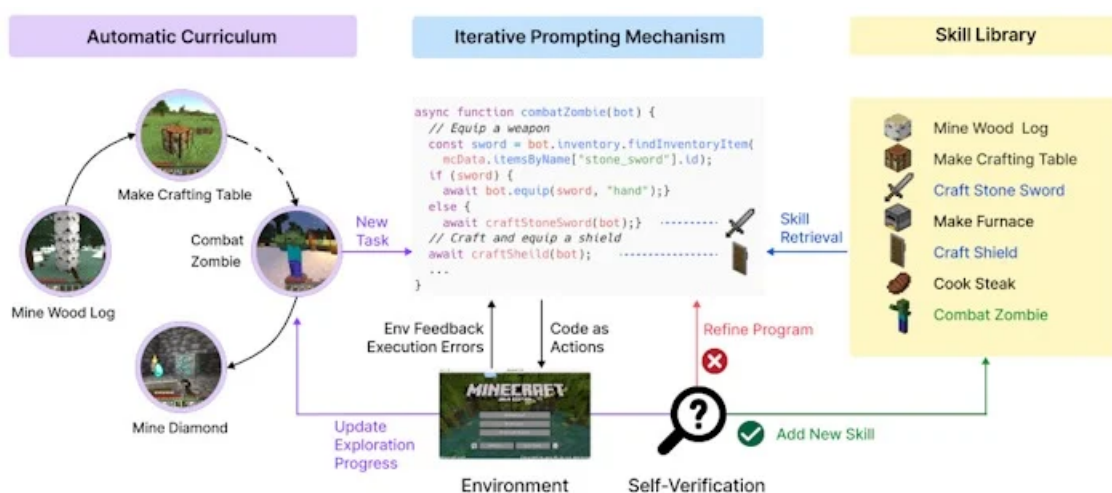
Dovednosti jsou přitom konstruovány hierarchicky. Složitější akce (např. „vytvoř železný krumpáč“) využívají jednodušší dovednosti jako „vypal železnou rudu“, která sama závisí na dovednosti „těž železnou rudu“ a „vyrob pec“. Tento kompoziční přístup umožňuje agentovi stavět na tom, co se již naučil, a minimalizuje nutnost začínat stále od nuly. Zároveň se tím eliminuje problém tzv. katastrofického zapomínání, který je běžný u běžných metod posilovaného učení.

V rámci samotného učení Voyager nevyužívá klasický přístup typu „pozorování → akce“, ale spoléhá na iterativní promptování. Při řešení úkolu agent nejprve navrhne program, který by měl daný úkol vyřešit. Tento program se následně provede v prostředí a výstup – včetně chyb a výjimek – je znovu předložen GPT-4. Na základě této zpětné vazby model prompt upraví, program opraví a zkusí jej znovu. Tento proces se opakuje, dokud tzv. sebe-verifikátor, opět založený na GPT-4, nevyhodnotí, že byl úkol úspěšně dokončen.

Iterativní přístup se ukázal jako klíčový zejména v případech, kdy agent narazí na nekonzistence mezi plánem a realitou. Například pokud program selže s chybou typu „I cannot make a stick because I need: 2 more planks“, systém je schopen automaticky provést korekci a aktualizovat plán. Pokud se problém opakuje (čtyři po sobě jdoucí neúspěšné pokusy), agent daný úkol opustí a vygeneruje si nový.

Ve srovnání s předchozími přístupy, jako jsou ReAct, Reflexion nebo AutoGPT, dosáhl Voyager výrazně lepších výsledků. V prostředí MineDojo získal $3,3\times$ více unikátních předmětů, prozkoumal svět do větší vzdálenosti ($2,3\times$ delší trasy) a dosáhl technologického pokroku až $15,3\times$ rychleji. Navíc jako jediný agent zvládl tzv. zero-shot úlohy, kde byl přenesen do nového světa s novým úkolem, který dosud neřešil – např. vytvoření dalekohledu nebo nalezení diamantů.

Díky kombinaci autonomního generování úkolů, kompoziční knihovny dovedností a schopnosti iterativní korekce chyb se Voyager přibližuje představě agenta, který je schopen dlouhodobého, otevřeného učení bez potřeby lidského zásahu. Schéma na obrázku 1.3 přehledně znázorňuje propojení těchto tří klíčových komponent. V prostředí, které není explicitně navrženo pro AI trénink, vykazuje chování, které je zároveň zvědavé, adaptivní i efektivní – a představuje tak významný krok k obecné umělé inteligenci v otevřeném světě.



Obrázek 1.3: Vizualizace propojení hlavních komponent systému Voyager.[26]

1.2 Některé z algoritmů a technik používané pro agenty

1.2.1 Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO)[7][9] je jedním z nejrozšířenějších algoritmů pro trénování agentů pomocí hlubokého posilovaného učení. Vyvinut byl výzkumníky ze společnosti OpenAI jako reakce na klíčové nedostatky tehdejších metod policy gradient algoritmů, které často provádějí příliš velké aktualizace politiky bez jakékoli kontroly, a složitější metody jako Trust Region Policy Optimization (TRPO)[8], které jsou výpočetně náročné a obtížně implementovatelné. PPO nabízí kompromis: spojuje jednoduchost implementace s vysokou stabilitou učení a efektivním využitím dat.

Algoritmus PPO patří do rodiny metod založených na optimalizaci politiky. Na rozdíl od metod hodnotové funkce, jako je Q-learning[13][21], přímo modeluje politiku agenta (pravděpodobnostní rozdělení nad akcemi) a učí se ji optimalizovat tak, aby maximalizoval dlouhodobou očekávanou odměnu. PPO navazuje na TRPO, který zavádí omezení velikosti aktualizace politiky, aby se zabránilo destruktivním skokům ve strategii agenta, ale činí tak výrazně jednodušším a výpočetně efektivnějším způsobem.

Základní myšlenka PPO spočívá v tom, že při každé aktualizaci politiky se zamezuje příliš velkým změnám pravděpodobnostních rozdělení nad akcemi. Konkrétně se zavádí tzv. *clipped surrogate objective*, která upravuje klasický výraz pro policy gradient, aby penalizovala změny mimo stanovený rozsah. Tento přístup umožňuje provádět více kroků stochastického gradientního vzestupu na stejných datech bez rizika destabilizace učení.

Matematicky lze optimalizovaný výraz popsat jako:

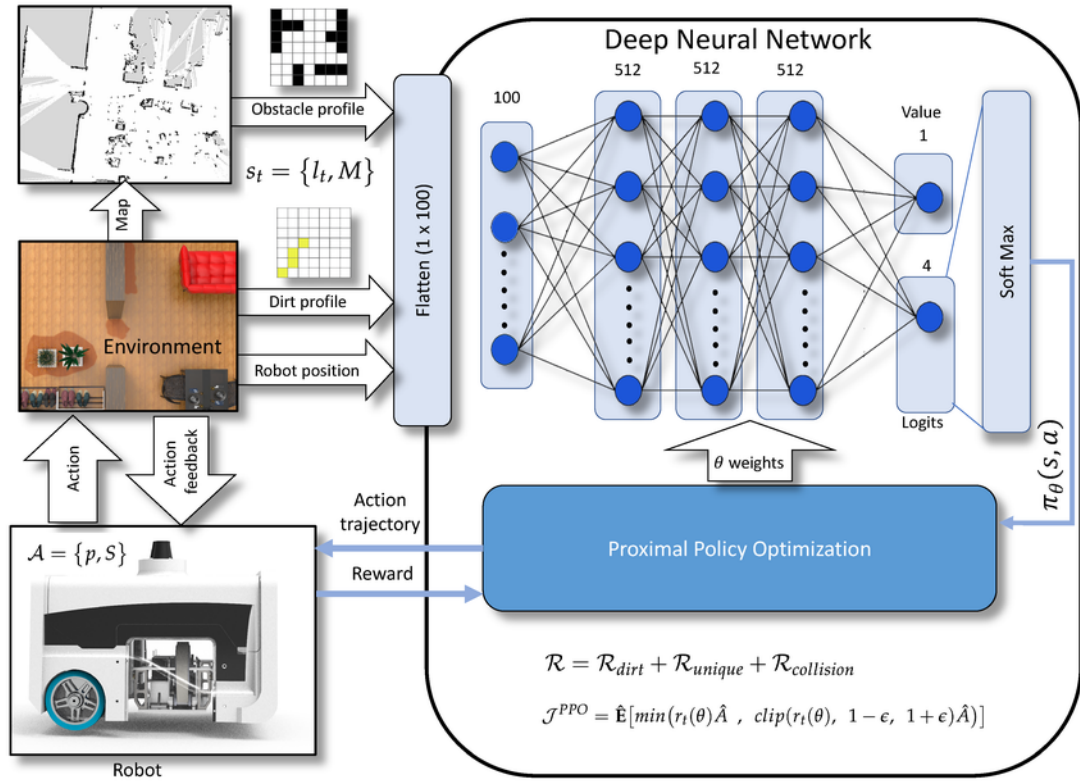
$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right],$$

kde

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$$

je poměr pravděpodobností podle nové a staré politiky a $\hat{A}_t$ je odhad tzv. advantage funkce, která kvantifikuje, jak výhodná byla daná akce vzhledem k očekávanému vývoji. Hodnota ϵ je hyperparametr, typicky nastavený na 0,1 až 0,3, který určuje toleranci změn politiky. Symbol $\hat{\mathbb{E}}_t$ představuje empirické očekávání vypočtené na základě vzorků nasbíraných během interakcí agenta s prostředím.

Smyslem výrazu *clip* je omezit motivaci agenta k tomu, aby příliš radikálně měnil svou strategii, pokud taková změna vede k výraznému zvýšení pravděpodobnosti akcí. Pokud je změna politiky příliš výrazná (např. $r_t > 1 + \epsilon$), je výpočet výsledku „oříznut“ tak, aby nevedl k nadhodnocení zisku. Naopak, pokud je změna v rámci povoleného pásma, je výpočet ponechán. Výsledkem je stabilnější učení, které se vyhýbá oscilacím a přetížení aktualizací. Celý proces je vizuálně shrnut na obrázku 1.4, který zachycuje architekturu PPO včetně interakce s prostředím, výpočtu hodnoty akce a aktualizace politiky.



Obrázek 1.4: Vizualizace architektury algoritmu PPO. Agent interaguje s prostředím, zatímco dvě neuronové sítě — actor a critic — slouží k výběru akcí a odhadu hodnoty stavu. Politika je aktualizována pomocí oříznutého objektivu a výhod jsou počítány pomocí Generalized Advantage Estimation (GAE).[27]

Kromě této hlavní ztrátové funkce lze PPO dále rozšířit o dvě další složky — penalizaci chyby odhadu hodnoty a entropickou regulaci. Celková ztrátová funkce pak má tvar:

$$L_t^{PPO}(\theta) = \mathbb{E}_t [L_t^{\text{CLIP}}(\theta) - c_1 L_t^{\text{VF}}(\theta) + c_2 \mathcal{S}(\pi_\theta, s_t)],$$

kde L_t^{VF} je kvadratická chyba hodnotové funkce, $\mathcal{S}(\pi_\theta, s_t)$ je entropie distribuční politiky ve stavu s_t a c_1, c_2 jsou váhové koeficienty. Přidání entropické složky pomáhá zajistit průzkum prostředí (exploration), zejména v počátečních fázích tréninku, a zabraňuje tomu, aby se agent „zasekl“ v lokálním maximu.

Pro výpočet výše zmíněné advantage funkce $\hat{A}_t$ se často používá metoda Generalized Advantage Estimation (GAE)[10]. Ta snižuje rozptyl odhadů tím, že kombinuje okamžité odměny a odhady hodnoty pomocí exponenciálního vážení parametrizovaného faktorem λ :

$$\hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \dots,$$

kde

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

a γ je diskontní faktor. GAE tak umožňuje lepší kompromis mezi zkreslením a rozptylem, což se v praxi promítá do rychlejšího a stabilnějšího učení.

Implementace PPO obvykle probíhá ve stylu tzv. *actor-critic*. Paralelní agenti (herní instance) generují trajektorie, typicky v rozsahu stovek časových kroků, a výsledné vzorky se následně použijí pro více iterací minibatch SGD nad ztrátovou funkcí. Při tréninku se často používá optimalizátor Adam a architektura s oddělenou politikou a hodnotovou funkcí, i když v některých případech lze využít i sdílené parametry.

Z praktického hlediska byl PPO úspěšně aplikován na širokou škálu prostředí. V oblasti spojitě kontroly (např. úkoly v prostředí MuJoCo[5]) překonal algoritmus PPO jiné tehdy běžně používané přístupy jako TRPO, A2C nebo CEM, a to jak z hlediska dosažené odměny, tak z hlediska rychlosti konvergence. Rovněž ve hrách z prostředí Atari, kde byl porovnáván s A2C a ACER, ukázal PPO vysokou robustnost napříč různými herními scénáři. Navíc oproti metodám s pevným KL penalizačním koeficientem dosahoval stabilnějších výsledků díky použití tzv. *adaptive KL penalty* nebo oříznutí podle výše uvedeného pravidla.

PPO se stal preferovaným algoritmem v mnoha aplikacích nejen díky výkonu, ale i díky snadné integraci do různých tréninkových rámců. Kromě empirických výsledků je výhodou PPO i jeho jednoduchost. Z hlediska implementace vyžaduje pouze mírnou úpravu klasického policy gradient frameworku a nevyžaduje řešení složitých optimalizačních úloh jako TRPO (např. pomocí konjugovaného gradientu). Díky tomu se stal PPO standardní volbou v mnoha open-source knihovnách posilovaného učení, včetně OpenAI Baselines, Stable Baselines nebo RLlib.

Celkově lze PPO vnímat jako rozumný kompromis mezi výpočetní náročností, robustností a výkonem. Jeho úspěch spočívá v jednoduché, ale účinné úpravě klasických policy gradient metod, která zajišťuje, že každá aktualizace politiky zůstane „blízko“ té původní, čímž se zamezí katastrofálním změnám a zároveň se umožní efektivní učení.

1.2.2 Deep Q-Network

Algoritmus Deep Q-Network (DQN)[21][22][23] je metoda posilovaného učení kombinující klasický Q-learning s hlubokými neuronovými sítěmi pro aproximaci akčně-hodnotové funkce $Q^*(s, a)$. Funkce $Q^*(s, a)$ udává očekávanou kumulovanou diskontovanou odměnu po provedení akce a ve stavu s a následném sledování optimální politiky. Teoretickým základem DQN je Bellmanova optimální rovnice:

$$Q^*(s_t, a_t) = \mathbb{E} \left[r_{t+1} + \gamma \max_{a'} Q^*(s_{t+1}, a') \mid s_t, a_t \right]$$

kde s_t je stav, a_t zvolená akce, r_{t+1} okamžitá odměna, s_{t+1} následující stav a $\gamma \in [0, 1]$ diskontní faktor. Klasický Q-learning provádí aktualizaci tabulky hodnot podle vztahu:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left(r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right)$$

kde α je rychlost učení. V prostředích s vysokodimenzionálním stavovým prostorem není tabulkové ukládání hodnot možné. DQN používá hlubokou neuronovou síť $Q(s, a; \theta)$ s parametry θ , která na vstupu přijímá stav s a na výstupu poskytuje vektor Q-hodnot pro všechny akce.

Trénink DQN minimalizuje časový rozdíl (TD error) mezi odhadovanou Q-hodnotou a cílovou hodnotou z Bellmanovy rovnice. Stabilizace učení využívá dvě hlavní techniky: přehrávací paměť (experience replay) a cílovou síť (target network). Přehrávací paměť $\mathcal{D}$ ukládá přechody $(s_t, a_t, r_{t+1}, s_{t+1})$ a umožňuje náhodné vzorkování mini-batchů, což přerušuje korelace a zlepšuje rozmanitost tréninkových dat. Cílová síť $Q(s, a; \theta^-)$ je kopie online sítě aktualizovaná periodicky, aby poskytovala stabilní cílové hodnoty.

Pro přechod v mini-batchi jsou cílová hodnota y_t a ztrátová funkce $L_t(\theta)$ definovány:

$$y_t = r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a'; \theta^-)$$

$$L_t(\theta) = \mathbb{E}_{(s, a, r, s') \sim \mathcal{D}} \left[(y_t - Q(s_t, a_t; \theta))^2 \right]$$

Parametry θ se aktualizují stochastickým gradientním sestupem:

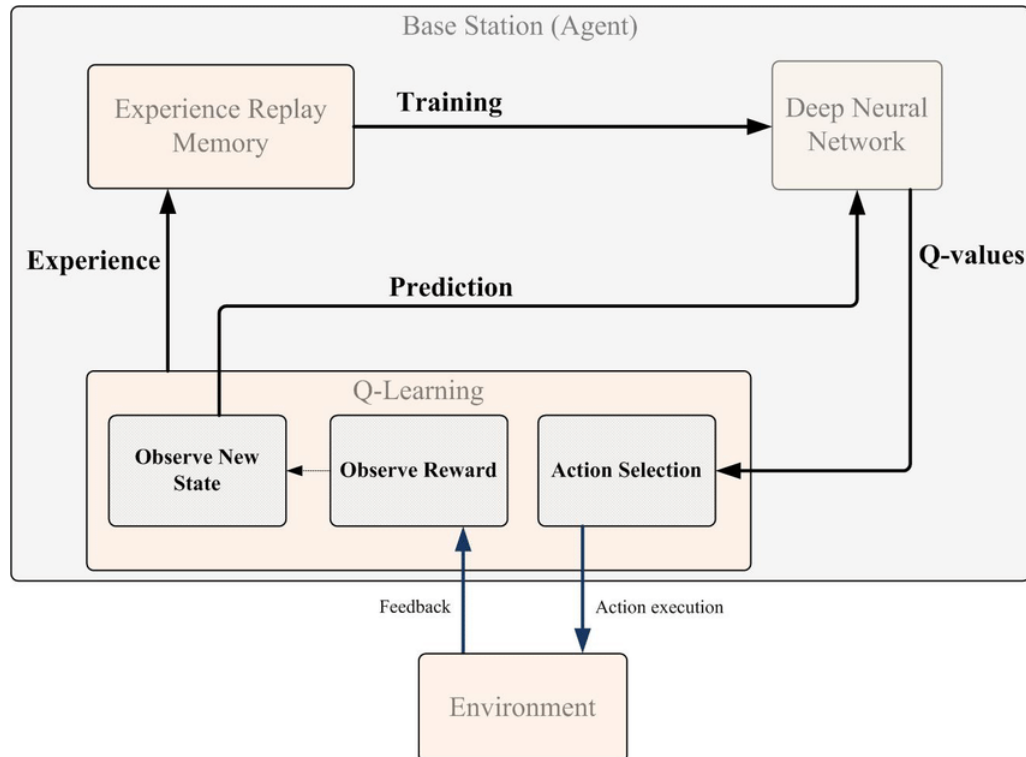
$$\theta \leftarrow \theta - \alpha \nabla_{\theta} L_t(\theta)$$

Váhy cílové sítě se synchronizují s hlavní sítí periodicky:

$$\theta^- \leftarrow \theta$$

Tréninkový cyklus zahrnuje sledování stavu s_t , výběr akce a_t pomocí ϵ -greedy politiky, provedení akce, získání odměny r_{t+1} a nového stavu s_{t+1} , uložení přechodu do paměti $\mathcal{D}$, vzorkování mini-batche, výpočet y_t , minimalizaci TD chyby a periodickou aktualizaci cílové sítě.

DQN je citlivé na nestabilitu a divergenci kvůli kombinaci nelineární aproximace neuronovou sítí a bootstrapovaných aktualizací. Přehrávací paměť redukuje autokorelaci dat a zvyšuje efektivitu využití zkušeností, cílová síť poskytuje stabilní referenční hodnoty. Pokročilé varianty zahrnují Double DQN, Dueling DQN a Prioritized Experience Replay, které zlepšují konvergenci, stabilitu a efektivitu učení v komplexních prostředích.



Obrázek 1.5: Příklad architektury Deep Q-Learning[30]

1.2.3 Self-Play

Self-play[16] je technika v oblasti posilovaného učení, která umožňuje agentům zlepšovat své strategie prostřednictvím interakce se sebou samými — tedy s kopiemi, staršími verzemi nebo současnými protivníky v rámci jednoho tréninkového rámce. Tato metoda se ukázala jako zásadní pro dosažení vyšších výkonů ve hrách, jako jsou Go, šachy, StarCraft II nebo Dota 2. Na rozdíl od klasického posilovaného učení, kde agent interaguje s předem daným prostředím, zde prostředí zahrnuje další agentní entity, jejichž chování se v čase mění — a tím se vytváří dynamický a nelineární tréninkový proces.

Základní přístup self-play spočívá v tom, že agent trénuje proti svým vlastním kopiím, ať už současným, nebo historickým. Nejjednodušší forma, tzv. „vanilla self-play“, spočívá v tom, že se agent neustále střetává s poslední verzí sebe sama. Tato metoda je velmi efektivní v tranzitivních hrách, kde lze jednotlivé strategie jednoznačně seřadit dle síly, avšak v nestranově tranzitivních prostředích (například kámen-nůžky-papír) vede často ke cyklickému chování a může konvergovat k suboptimálním strategiím.

Aby se tyto nedostatky zmírnily, vznikly složitější varianty. „Fiktivní self-play“ rozšiřuje trénink tak, že agent hraje proti průměru všech svých předchozích verzí. Tím se posiluje robustnost učení a eliminuje riziko zacyklení. „Prioritizovaná fiktivní self-play“ (např. použitá v AlphaStar) dále zavádí vážený výběr protivníků, čímž zvýhodňuje ty, proti nimž si agent vede hůře — tedy záměrně trénuje proti „slabým místům“ své strategie.

Další variantou je „ δ -uniform self-play“, která používá hyperparametr δ k určení, jaké části historické populace budou použity jako soupeři. Nižší δ znamená širší spektrum historických politik, vyšší hodnoty naopak preferují nedávné soupeře.

Moderní rámce self-play algoritmů se často integrují s tzv. „population-based“ přístupy. Místo jednoho agenta existuje populace politik Π , která se v čase rozšiřuje o nové verze. Každá nová politika je trénována vůči vzorku soupeřů, vybíraných z Π na základě určité metastrategie Σ , často reprezentované interakční maticí. Tento proces vytváří bohatý prostor pro evoluci strategií a vzájemnou adaptaci.

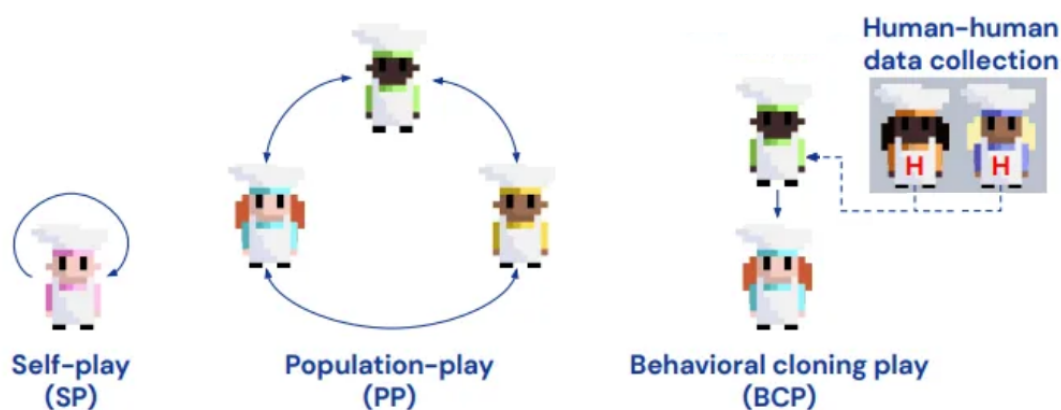
Základní tréninková smyčka obvykle probíhá následovně:

1. Nové politiky jsou inicializovány, často na základě posledních iterací předchozích politik.
2. Každá politika je trénována proti soupeřům vybraným podle Σ .
3. Nově natrénované politiky se přidají do populace a proces se opakuje.

Z hlediska výpočetní efektivity se rozlišuje mezi tzv. „lazy initialization“, kdy se politiky inicializují až v okamžiku potřeby, a „immediate initialization“, kdy je celá populace vytvořena hned na začátku. Dále rozlišujeme „standardní trénink“, kde se v každé iteraci trénuje pouze jeden agent, a „průběžný trénink“ (např. FTW agent), kde jsou všechny politiky trénovány souběžně.

Zvláštní pozornost si zaslouží tzv. PSRO (Policy-Space Response Oracles), které rozšiřují klasické self-play o výpočet rovnovážných strategií v rámci tzv. meta-herního prostoru. Místo toho, aby se agent učil proti jednotlivým politikám, PSRO používá například Nashovu rovnováhu jako metastrategii, vůči níž se hledají nejlepší odpovědi.

Kromě klasických her našla self-play široké uplatnění i v komplexních videohrách a reálných scénářích, například v robotice, autonomním řízení nebo syntéze proteinů. Také se používá, zatím v menší míře, v kontextu velkých jazykových modelů, například pro sebezdokonalování (self-improvement) nebo samostatnou konstrukci výukových dat. Základní rozdíl mezi přístupy self-play, population-play a behavioral cloning je naznačen na obrázku 1.6.



Obrázek 1.6: Srovnání tří přístupů k tréninku agentů: „self-play“ (SP), kde agent hraje sám proti sobě; „population-play“ (PP), kde interaguje s různými agenty v populaci; a „behavioral cloning play“ (BCP), kde se učení zakládá na lidských demonstračních datech. Upraveno podle zdroje[28]

1.2.4 Population Based Training

Population-Based Training (PBT)[4] je technika, která propojuje samotný trénink neuronových sítí s automatickým laděním hyperparametrů. Místo toho, aby se model trénoval s pevně danou konfigurací a po dokončení se hledala optimální nastavení v dalších bězích, PBT tento proces spojuje — optimalizuje váhy i hyperparametry zároveň a průběžně. To je zvláště užitečné v oblastech, kde jsou optimální hyperparametry v čase proměnlivé, například u posilovaného učení nebo generativních modelů.

Na začátku PBT inicializuje populaci modelů, kde každý má své vlastní váhy i hyperparametry. Všichni členové této populace jsou trénováni paralelně a nezávisle — často metodami jako gradient descent. Po určitém počtu kroků, nebo jakmile model dosáhne dostatečného výkonu, probíhá tzv. „exploit-and-explore“ fáze. Pokud je některý z modelů podprůměrný, může být jeho obsah nahrazen jiným, výkonnějším modelem z populace. Zároveň se na něj aplikuje lehce pozměněná verze jeho hyperparametrů — čímž vzniká nový „směr“ učení.

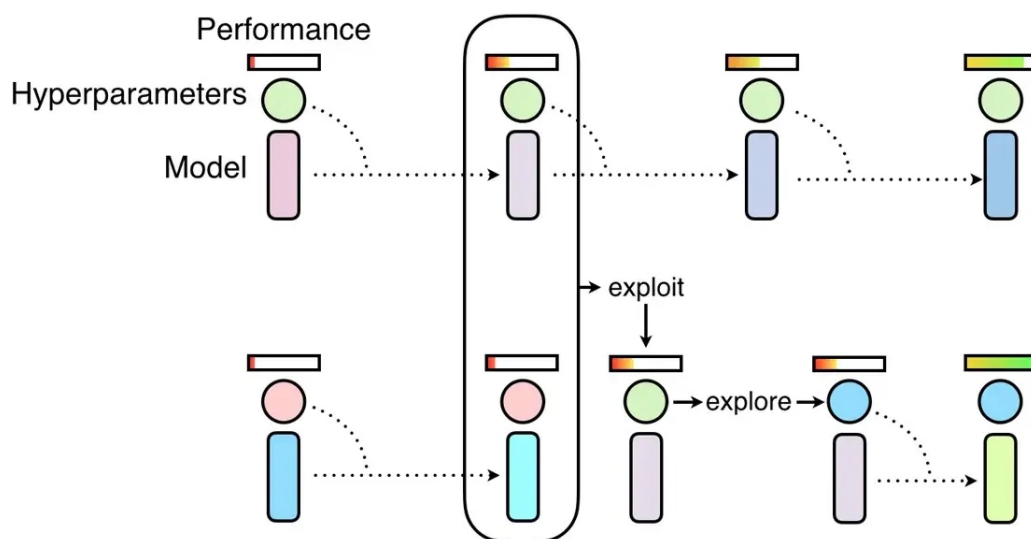
Tento přístup vytváří smyčku, ve které probíhá klasický trénink modelů, ale průběžně se mezi sebou také sdílejí informace a testují se nové kombinace. Tím dochází nejen k rychlejšímu konvergenci, ale také ke zlepšení finálního výkonu, protože hyperparametry nejsou fixní, ale průběžně se přizpůsobují aktuálnímu stavu tréninku.

Z praktického hlediska je PBT velmi dobře škálovatelný. Nevyžaduje centrální koordinaci — modely komunikují jen skrze jednoduché sdílení výkonových metrik a checkpointů. Díky tomu je možné PBT provozovat i na heterogenních nebo nestabilních výpočetních platformách, například v cloudovém prostředí s preemptibilními instancemi.

Základní smyčka algoritmu vypadá následovně:

1. Každý model ve své instanci trénuje standardními optimalizačními metodami (například SGD nebo Adam) pomocí aktuální konfigurace hyperparametrů.
2. Po určité době se jeho výkon vyhodnotí (například pomocí validace nebo herní odměny).
3. Pokud je model označen jako „ready“, provede se fáze „exploit“ — model si může zkopírovat váhy a hyperparametry od lepšího modelu v populaci.
4. Poté následuje fáze „explore“, kde jsou hyperparametry lehce pozměněny (například násobkem 1.2 nebo 0.8), případně náhodně přegenerovány.
5. Trénink pokračuje s těmito novými parametry.

Tento proces je znázorněn na obrázku 1.7, kde je vidět, jak modely mezi sebou sdílejí váhy a mění hyperparametry na základě výkonnosti.



Obrázek 1.7: Population-Based Training neuronových sítí začíná jako náhodné prohledávání, ale v průběhu tréninku umožňuje jednotlivým modelům využít dílčí výsledky ostatních (fáze „exploit“) a zároveň zkoušet nové kombinace hyperparametrů (fáze „explore“)./citePBT2017

Důležité je, že PBT není citlivý na přesné nastavení toho, kdy se fáze „exploit-and-explore“ provádí — obvykle stačí, aby mezi dvěma takovými událostmi uplynulo dost kroků na to, aby gradientové učení mělo čas ukázat efekt.

Technika PBT byla úspěšně aplikována v několika odlišných oblastech. V oblasti posilovaného učení pomohla například zrychlit trénink agentů v prostředí DeepMind Lab, StarCraft II nebo Atari. V těchto případech se automaticky ladily parametry jako learning rate, entropická penalizace nebo délka rozvinutí sítě (unroll length). V oblasti strojového překladu pomocí Transformerů PBT automaticky našla hyperparametrové rozvrhy, které překonaly ručně laděné baseline modely. A u GANů PBT dokázala zlepšit stabilitu tréninku i finální kvalitu generovaných obrázků.

Klíčovou výhodou PBT oproti klasickým metodám ladění (jako je grid search nebo Bayesian optimization) je, že probíhá „online“ — tedy současně s tréninkem. Díky tomu je výrazně efektivnější, protože modely nejsou trénovány od začátku znovu po každé změně hyperparametrů. PBT tak nabízí kombinaci výhod klasického evolučního přístupu a gradientového učení.

Celkově lze PBT vnímat jako techniku, která vnáší do optimalizace adaptivitu a sdílení znalostí v reálném čase. Je to metoda, která se výborně doplňuje s dalšími technikami, jako je self-play.

Kapitola 2

Použité technologie a návrh hry

2.1 Použité technologie

2.1.1 Unreal Engine

Prostředí, ale i agent budou vytvářeni a použiti v Unreal Engine 5.2, který zajišťuje mapu i herní mechanismy s moderními renderovacími funkcemi v reálném čase. Některé z použitých prvků ve hře (hlavně modely a animace) pocházejí z FABu – nového tržiště od Epic Games, které spojilo staré Unreal Marketplace, Sketchfab, Quixel a ArtStation v jednu online platformu.

Unreal Engine (UE)[17], vyvíjený společností Epic Games, je výsledkem evoluce začínající v roce 1998 a dnes představuje špičkový engine pro vytváření her, filmů či vizualizací. Jeho pružnost, nastavitelnost a výkon jsou velmi ceněné mezi vývojáři po celém světě.

Velkou předností UE je systém Blueprint vizuálního skriptování, což je nadstavba C++, která umožňuje vytvářet herní logiku, interakce nebo AI pomocí uzlů a grafů místo čistého C++. Díky tomu je tvorba her dostupnější i kreativcům, kteří nejsou programátoři. A pokud jsou jednotlivé uzly přehledně poskládány, může být práce s nimi dokonce rychlejší pro prototypování a nasazení systémů než při použití čistého C++.

Spíše UE5 než UE obecně si získal oblibu nejen mezi vývojáři her, ale i ve filmové produkci, architektonické vizualizaci, produktovém designu nebo v průmyslovém a tréninkovém softwaru. Filmoví tvůrci ho využívají pro virtuální natáčení v simulovaných prostředích, architekti jej nasazují pro interaktivní prezentace klientům a byl také využit v lékařství či obraně ke školení zaměstnanců ve vysoce realistických scénářích.

Důvody, proč byl zvolen Unreal Engine před alternativami jako je Unity, jsou následující:

1. C++:

- Jazyk C++ je jeden ze dvou použitých jazyků v Unreal Enginu. Tento jazyk je velice rychlý, což je u aplikací, jako jsou např. hry běžící v reálném čase, potřebné. V tomto jazyce mám zároveň nejvíce zkušeností oproti např. C#, které používá Unity.

2. Programovací nadstavba Blueprint:

- Jak už bylo zmíněno výše, Blueprint je nadstavba pro programovací jazyk C++ a velice zjednodušuje programování v UE, což znamená, že ho dokáže používat téměř kdokoli, i s minimálními programovacími schopnostmi.

3. FAB:

- Také bylo výše zmíněno, že dříve byly Unreal Marketplace, Sketchfab, Quixel a ArtStation samostatná tržiště/programy, kde jste na Unreal tržišti mohli sehnat kód nebo modely, Sketchfab vám umožňoval upravovat modely, Quixel je něco mezi Unreal tržištěm a Sketchfabem a ArtStation slouží také k vyhledávání a získávání modelů. Tyto čtyři obchody byly spojeny v jeden pro Epic Games pod jménem FAB. Díky FABu nebylo zapotřebí vytvářet téměř žádné modely, protože většina modelů, které byly použity ke tvorbě mapy, hráčské postavy, zbraní atd., pochází z FABu. Toto dokáže mnohonásobně zrychlit vývoj her.

4. Pluginy:

- Pokud je zapotřebí rozšířit Unreal Engine o funkce, které neobsahuje v základním balíčku, lze přidat různé pluginy pomocí obchodu s pluginy, nebo také přidáním pluginu přímo do složky, kde je Unreal Engine nainstalovaný.

5. 3D:

- UE je navržený od základů pro 3D tvorbu. V Unity nebo např. Godotu lze vytvářet 3D hry, ale jejich tvorba je mnohem náročnější než v UE, a kvůli tomu se tyto nástroje spíše hodí pro 2D nebo 2.5D projekty.

Unreal Engine má ale i své nevýhody oproti např. Unity nebo dokonce i Godotu:

1. Menší komunita:

- Unity má stále širší uživatelskou základnu oproti Unreal Enginu, takže někdy může být těžší najít odpovědi na specifické problémy v UE, než by bylo v Unity. Godot je na tom ale podobně jako UE.

2. Náročnější nároky na výkon:

- Na rozdíl od Unity, které dokáže být celkem svižné s CPU, který má 6 jader, 16 GB RAM a GTX 1060, Unreal Engine 5 pro dobrý výkon ideálně vyžaduje minimálně 32 GB RAM, ale spíše doporučované minimum je 64 GB RAM, moderní vícejádrové CPU a minimálně grafickou kartu RTX 3080, ideálně však RTX 4080 nebo RTX 4090 pro real-time renderování. Na stanicích, které používá Epic Games pro UE, se běžně vyskytují konfigurace se 128 nebo dokonce 256 GB RAM, procesory Threadripper a RTX 4080 pro zpracování rozsáhlých scén bez kompromisů.



Obrázek 2.1: Ikony použitých technologií

2.1.2 Visual Studio

V tomto projektu používám pro programování v C++ vývojové prostředí Visual Studio, které slouží k psaní kódu v tomto jazyce. Unreal Engine 5 přímo vyžaduje Visual Studio pro jakoukoliv práci s C++, protože je s tímto prostředím úzce propojen. Bez něj nelze kompilovat ani spouštět vlastní, čistě C++ moduly.

Visual Studio je komplexní integrované vývojové prostředí (IDE) vytvořené společností Microsoft. Ačkoli podporuje více programovacích jazyků, při práci s Unreal Enginem se využívá především pro C++. Nabízí širokou škálu nástrojů – od správy projektů, přes inteligentní doplňování kódu (IntelliSense), až po ladění a analýzu paměti.

Při práci s Unreal Enginem se Visual Studio stáhne a automaticky nakonfiguruje pro správu projektové struktury a závislostí. Využívá build systémy jako MSBuild nebo CMake a přímo spolupracuje se soubory Unreal projektu, což zajišťuje, že nové C++ třídy a funkce budou správně sestaveny a propojeny s editorem.

Visual Studio rovněž umožňuje ladění v reálném čase, profilování výkonu a sledování využití paměti – což jsou klíčové nástroje při vývoji rozsáhlých projektů v UE5.

Při vývoji na Windows je Visual Studio pro Unreal prakticky nezbytné. Umožňuje efektivní práci s C++, řízení kompilace a plynulý vývojářský proces při programování na úrovni enginu.

2.1.3 NevarokML

NevarokML[18] je pokročilý plugin pro Unreal Engine 5, který umožňuje přímou integraci strojového učení – konkrétně reinforcement learningu (RL) – do herních nebo simulačních projektů. Plugin je postaven na knihovně Stable-Baselines3 a zpřístupňuje populární RL algoritmy jako PPO, A2C, DDPG, DQN, SAC a TD3, které agentům umožňují učit se optimálnímu chování metodou pokus–omyl.

Podporována je jak jednoagentní, tak víceagentní interakce, včetně možnosti paralelního tréninku v různých prostředích, čímž se výrazně zrychluje učení a zvyšuje schopnost agentů generalizovat v různých scénářích.

Architektura NevarokML funguje na principu modelu klient–server: Unreal Engine běží jako serverová část, která simuluje prostředí a chování agentů, zatímco pythonový klient provádí samotné strojové učení pomocí algoritmů ze Stable-Baselines3.

Komunikace mezi Unreal Enginem a Python klientem probíhá přes síť nebo sockety pomocí JSON struktury, která přenáší stav prostředí (pozorování, odměny, akce). Jakmile je model natrénován, výsledné váhy se převedou do formátu ONNX a následně se jako objekt typu NNEModelData nasadí přímo do Unreal Enginu prostřednictvím jeho systému Neural Network Engine.

ONNX[19], neboli Open Neural Network Exchange, je otevřený formát a celý ekosystém navržený tak, aby umožnil snadný přenos již natrénovaných modelů hlubokého učení mezi různými vývojovými frameworky, platformami a typy hardwaru – a to bez potřeby složité nebo zdoluhavé konverze.

Neural Network Engine (NNE)[20] je plugin navržený pro provádění inferencí neuronových sítí v reálném čase přímo v prostředí Unreal Engine. Využívá formát ONNX, který umožňuje spouštění modelů exportovaných z běžně používaných frameworků strojového učení, jako jsou PyTorch, TensorFlow nebo MXNet, ve formátu .onnx.

Jak vypadá základní pracovní postup:

1. Vytvoření prostředí:

- Začíná se vytvořením Blueprint třídy děděné z ANevarokMLEnv, kde se definují základní události OnInit, OnStep, a OnReset pro řízení logiky prostředí a průběhu epizod.

2. Trenérský modul:

- Poté se vytvoří Blueprint děděný z ANevarokMLTrainer, kde se nastaví pozorovací a akční prostor (OnConstruct) a zahájí samotný trénink (OnStart).

3. Implementace v úrovni:

- Trenér a prostředí se vloží do Unreal scény, kde trenér odkazuje na prostředí a celý proces tréninku začne při spuštění levelu.

4. Sledování učení:

- Pro vizualizaci průběhu tréninku lze použít TensorBoard, který umožňuje sledovat vývoj odměn, změny v politice agenta nebo ladění hyperparametrů.

Plugin je dostupný pro Unreal Engine verzi 5.2, a po aktivaci pluginu je nutné zprovoznit i jeho backend klienta a zkontrolovat připojení mezi Unreal a Pythonovým prostředím.

Hlavní vlastnosti a využití:

1. Široký výběr algoritmů:

- Podpora různých typů RL algoritmů umožňuje ladit chování agentů podle konkrétního použití (např. hodnotové nebo policy-gradient přístupy).

2. Podpora Blueprints i C++:

- Plugin je plně využitelný jak přes vizuální skriptování, tak i přes nativní C++ rozhraní pro pokročilé zásahy.

3. Cross-platform nasazení:

- I když je trénink typicky prováděn na Windows, natrénované modely lze nasadit na jakoukoliv platformu podporovanou Unreal Enginem.

4. Vhodné pro výuku a výzkum:

- Plugin je volně dostupný pro nekomerční využití, což z něj činí ideální nástroj pro akademické projekty, simulace nebo vývoj prototypů.

2.2 Návrh hry

2.2.1 Návrh prostředí hry

Prostředí, ve kterém bude autonomní agent existovat, může být realizováno různými způsoby – od jednoduchých až po technicky náročnější. Mezi přístupnější možnosti patří využití herních engineů, jako je Unity nebo Unreal Engine, zatímco pokročilejší varianta zahrnuje programování od základu pomocí nízkourovňových API, jako je OpenGL nebo DirectX.

V tomto projektu bude vizualizace agenta realizována v Unreal Engine formou střelčky z pohledu první osoby. Prvním krokem bude vytvoření herní mapy, která umožní volný pohyb hráče i postavy řízené umělou inteligencí. Mapa bude inspirována městským náměstím po bitvě – její střed bude tvořit park s bankovní budovou. Park bude téměř prázdný kvůli zpusťování a bankovní budova bude obklopena bariádami. Právě v této budově se hráč po spuštění hry objeví. Nacházet se zde budou zbraně a box se zbraněmi a brněním, které hráč využije v boji.

Kromě parku bude mapa obsahovat i zastavěnou oblast – uzavřenější prostor obklopený budovami, jenž bude představovat vyšší riziko kvůli omezené viditelnosti a pohybu. Uvnitř zabarikádované zóny se budou nacházet výchozí body (spawn pointy) pro nepřátelské postavy ovládané AI.

Pro vyvážení obtížnosti budou nepřátelé po smrti odhazovat sbíratelné předměty, jako jsou lékárničky a zásobníky munice, které hráč využije k doplnění zdraví nebo střeliva.

Součástí uživatelského rozhraní bude tzv. HUD (Head-Up Display), jenž bude zobrazovat klíčové herní informace – aktuální stav zdraví, zásoby energie, počet nábojů a další. Z tohoto rozhraní bude dostupné jak hlavní menu, tak i pauzovací nabídka.

Celá hra bude mít strukturu založenou na kolových vlnách. Po spuštění prvního kola se dveře banky zavřou, ale venku bude umístěna další bedna s vybavením, kde bude možné měnit zbraně a brnění. V každém kole bude úkolem hráče eliminovat stanovený počet nepřátel. Po jejich porážce se aktuální kolo uzavře a automaticky začne nové – s vyšší obtížností nebo větším počtem protivníků. Tím vznikne dynamický cyklus, který prověří schopnosti hráče i chování autonomního agenta v různých podmínkách.

Část prostředí a kódu u různých blueprintů byla převzata z jednoho z mých projektů, ale proběhly zde velké změny, takže v další kapitole bude popis jednotlivých částí hry a poté agenta.

2.2.2 Návrh agenta

Cílem je vytvořit agenta řízený pomocí technik strojového učení, který se dokáže samostatně orientovat v herním prostředí, přiblížit se ke svému cíli (hráči) a efektivně použít střelnou zbraň k jeho eliminaci. Tento agent nebude řízen předem nadefinovanými skripty, ale bude se učit chování na základě zkušenosti pomocí posilovacího učení. Učící algoritmus bude založen na metodě Proximal Policy Optimization (PPO). Aby měl agent dostatečný přehled o situaci, budeme mu prostřednictvím funkce `CollectObservations` (sbírej pozorování) muset předávat soubor dat, ze kterého bude moct čerpat. Tyto informace umožní agentovi učinit informované rozhodnutí o tom, jaké akce má provést.

Agent bude mít přístup ke třem základním typům akcí:

- Pohyb
- Rotace
- Střelba, pokud jsou splněny podmínky (např. není aktivní cooldown, kouká na nepřítele atd.)

Tyto akce budou implementovány ve funkci `ApplyActions` (aplikuj akce), která interpretuje výstup neuronové sítě během trénování i při použití.

Jádrem efektivního tréninku bude systém odměn, implementován ve funkci `HandleRewards`, který určuje, jakým způsobem bude agent penalizován či odměňován za své chování. Musí dostat odměny za to, že bude mít správnou rotaci, směr pohybu nebo optimální vzdálenost a musí být limitován tak, aby agent nebyl motivován jen farmit body bez reálného splnění cíle. Veškerá motivace bude vázána přímo na výsledek boje.

Na základě toho návrhu se očekává, že agent:

1. Rozpozná pozici cíle pomocí poskytnutých pozorování.
2. Přiblíží se na efektivní vzdálenost.
3. Srovná rotaci tak, aby byl správně natočen k cíli.
4. Zahájí palbu a pokusí se cíl eliminovat.

Tato posloupnost chování by měla vzniknout emergentně díky optimalizaci očekávané kumulativní odměny. Agent si taky bude moct po každém kole si přizpůsobit svoje vybavení vůči vybavení, které měl minule kolo hráč, aby měl vyšší šanci ho porazit.

Kapitola 3

Implementace hry a agenta, vylepšení jeho algoritmu a porovnání verzí

3.1 Základní logika hry

3.1.1 Hráčská postava

Po spuštění hry se aktivuje úvodní funkce, která inicializuje několik klíčových pod-systémů hráčské postavy. Nejprve se připojí mapa ovládacích vstupů, která umožňuje hráči ovládat pohyb postavy pomocí klávesnice. Následně se k postavě připojí tzv. průhledový displej (HUD), díky kterému se zobrazují herní informace na obrazovce. Součástí této inicializace je také aktivace zaměřovače, který se synchronizuje s HUDem a reaguje na hráčovy vstupy. Další funkce aktivují systémy obnovování – konkrétně regeneraci života, energie a baterie svítilny. Zároveň se zpřístupní sekce HUDu, které tyto parametry zobrazují. Následující část vytváří proměnné potřebné pro propojení postavy v blueprintu s logikou sběru předmětů a obsluhou zbraní. Závěrečná inicializační funkce rozšiřuje HUD o prvek zobrazující typ právě vybavené zbraně.

Pohyb hráčovy kamery je řízen pomocí pohybu myši – horizontálně po ose Z a vertikálně po ose Y. Samotný pohyb postavy po mapě se realizuje zvlášť pro směr vpřed/vzad a pro pohyb do stran. V obou případech se využívají směrové vektory a výsledná hodnota se předává do funkce pro fyzický posun postavy. Systém zároveň detekuje, zda hráč běží či se plíží, aby mohl spustit nebo zastavit příslušné zvukové efekty.

Pro plynulé ozvučení pohybu je využito makro, které opakovaně volá funkci zajišťující přehrávání kroků. Pokud hráč přestal padat (například po skoku), znovu se aktivuje zvuk chůze. Systém zároveň detekuje typ povrchu pod nohama pomocí trasovací linky – podle nalezeného materiálu se přehrává odpovídající zvuk kroků.

Běhání je vázáno na dostupnou energii. Pokud hráč neběží a není v podřepu, ale začne se pohybovat a má dostatek energie, hra aktivuje běh. Energie se začne postupně snižovat a změní se rychlost přehrávání zvuku chůze. Jakmile běh ustane, energie se opět pomalu doplňuje a rytmus zvuku se přizpůsobí.

Podobně je řešen i úskok do stran. Pokud má hráč více než 15 bodů energie a není v podřepu, může provést úskok ve směru aktuálně stisknuté pohybové klávesy. Po dokončení úskoku se po krátké pauze začne energie obnovovat.

Při plížení (podřepu) systém kontroluje, zda hráč neběží ani neuskočil a zda má dostatek stamina. Pokud ano, postava se přepne do podřepu, tempo chůze se zpomalí a začne se doplňovat energie. Při návratu do vzpřímené pozice se část energie spotřebuje, ale opět začne regenerace.

Součástí logiky hráčské postavy je také systém pozastavení hry. Po stisknutí příslušného tlačítka se hra přeruší a na obrazovce se zobrazí menu pauzy.

Mechanismus spotřeby a regenerace energie (staminy) je řízen samostatnými funkcemi. Během běhu nebo jiných fyzicky náročných akcí se každým tikem odečítá stanovené množství stamina. Jakmile její hodnota klesne na nulu, postava přestane běžet. Naopak, pokud hráč není aktivně v pohybu, aktivuje se obnova energie, která ji postupně přičítá až do maximální hodnoty.

Postava má také funkce pro správu zdraví. Pokud je hráč zraněn, aktivuje se odečet životů. Následně, pokud postava stále žije, začne pomalá regenerace zdraví. Jakmile hodnota života dosáhne maxima, regenerace se pozastaví. Pokud však zdraví klesne na nulu, postava umírá – ovládání se zablokuje, zvuky se vypnou a zobrazí se obrazovka smrti. Odtud má hráč možnost hru ukončit nebo začít od začátku.

3.1.2 Zbraňový systém

Zbraňový systém ve hře je navržen tak, aby simuloval chování střelných zbraní, včetně střelby, přebíjení, sběru munice a interakce s prostředím. Na začátku se vytvoří ukazatel na hráče, což umožňuje přímou interakci mezi zbraní a hráčskou postavou.

Při výstřelu se automaticky odebere jeden náboj ze zásobníku a spustí se dvě animace – jedna vizualizuje samotný výstřel zbraně, druhá animaci hráče při střelbě. Pokud jsou náboje v zásobníku vyčerpány, zbraň začne vydávat charakteristický zvuk prázdného cvaknutí, který hráče upozorní na nutnost přebít.

Přebíjení se spustí pouze tehdy, pokud zásobník není plný. Systém nejprve zjistí, kolik nábojů chybí do plné kapacity zásobníku, a poté doplní chybějící náboje z celkové zásoby. Tento proces je doprovázen přehráním animace přebíjení. Pokud hráč na mapě sebere náboje, ověří se, zda ještě není překročen maximální počet. Pokud ne, náboje se přičtou do celkové zásoby.

K interakci se zbraněmi na mapě slouží dvě specifická tlačítka – tlačítko E pro sběr zbraní ze země a tlačítko B pro jejich odhození. Při sběru zbraně hra detekuje její přítomnost v okolí hráče, načte informace o typu zbraně, aktuálním počtu nábojů v zásobníku a celkové zásobě, a tyto údaje převede do inventáře hráče. Zbraň se následně odstraní ze světa a nahradí ji aktivní zbraň v ruce hráče, která je vizuálně připojena k jeho postavě.

Při odhození zbraně se zbraň v ruce hráče zničí a na místě před hráčem se vytvoří nová zbraň, kterou je možné znovu sebrat.

Logika střelby zohledňuje typ zbraně. Pokud má hráč v ruce automatickou pušku, hra aktivuje opakovanou střelbu, dokud jsou v zásobníku náboje. U brokovnice je implementována funkce náhodného rozptylu projektilů, která simuluje široký zásehový úhel typický pro tento typ zbraně. Brokovnice vystřelí několik střel najednou, každou s mírně odlišnou trajektorií. V případě odstřelovací pušky se spouští klasická střelba s jedním přesným výstřelem.

Střelba je iniciována funkcí, která detekuje rotaci a pozici hráčovy kamery. Ze zbraně je vyslána příčka simulující trajektorii střely, která může ovlivnit nepřátelskou umělou inteligenci. Další paprsek vychází přímo z kamery hráče a končí ve vzdálenosti určené směrem pohledu. Simulace rovněž zohledňuje nepřesnosti způsobené pohybem postavy a náhodný rozptyl u brokovnice.

Po výstřelu dochází k simulaci zpětného rázu, která upraví směr pohledu hráče. Pokud je zasažen nepřítel, aplikuje se poškození. V případě zásahu objektu s fyzikální interakcí se na něj aplikuje síla v opačném směru vůči směru střely.

Každý výstřel také spustí zvukový event, na který mohou nepřátelé reagovat. Zbraň rovněž generuje efekt vyhození prázdné nábojnice – ta po výstřelu vyletí ze zbraně a po určité době automaticky zmizí ze scény.

Tlačítko pro přebíjení (R) volá funkci, která spustí výše popsany proces doplnění zásobníku, čímž je zajištěna plná integrace mezi hráčovými vstupy a systémem správy munice.

Celý zbraňový systém je pečlivě propojen s HUDem, který hráči poskytuje vizuální zpětnou vazbu o typu zbraně, počtu nábojů a stavu zásob.

Agent využívá podobný zbraňový systém, který je však zjednodušený – nevyužívá všechny funkce ani některé modifikátory.

3.1.3 Logika řízení herních kol

Logika řízení herních kol a nepřátelských jednotek ve hře začíná inicializací základních proměnných. Po spuštění úrovně se nastaví ukazatel na hráče, obnoví se hlasitost zvukových efektů podle posledního uloženého nastavení a zavolá se funkce pro zahájení nové herní seance.

V rámci této funkce se resetuje proměnná určující, zda již byli všichni nepřátelé naspawnováni, inkrementuje se čítač aktuálního kola a spustí se časovač, který po uplynutí definované doby aktivuje začátek nového kola.

Zahájení kola zahrnuje nastavení počtu nepřátel, kteří se mají v daném kole objevit. Poté se spustí časovač, který s definovaným zpožděním volá funkci pro výběr náhodného místa spawnování. Na tomto místě se následně vytvoří nová instance nepřítele. Každý nepřítel je implementován jako trenér neuronové sítě, který je součástí tréninkové logiky.

Každý spawnovaný nepřítel je vytvořen na základě pozice a orientace předem definovaných spawnovacích bodů, čímž se zajišťuje variabilita a nepředvídatelnost jednotlivých kol.

Paralelně běží neustále se opakující kontrolní funkce, která monitoruje počet aktivních nepřátel. Jakmile systém detekuje, že byli všichni nepřátelé eliminováni, automaticky aktivuje ukončení aktuálního kola.

Závěrečná funkce každého kola zjišťuje, jakou zbraň měl hráč v ruce a jaké měl nastavené brnění (logika brnění je popsána dále v kapitole), a následně spouští další kolo.

3.1.4 Systém sbírání předmětů

Logika sběru herních předmětů je založena na detekci kolize mezi hráčovou postavou a specifickou kolizní sférou umístěnou kolem sbíratelných objektů. Jakmile dojde ke kontaktu těchto dvou komponent, spustí se funkce, která aktivuje možnost interakce a zvednutí daného objektu. Pokud hráč oblast opustí, volání funkce se přeruší a předmět zůstane nedotčen.

Při sběru zbraní kolizní sféra kromě detekce kontaktu uchovává také informace o konkrétní zbraní – například její typ a počet nábojů v zásobníku i celkové zásobě. Po stisknutí příslušné klávesy předá sféra tyto údaje hráčovu blueprintu, čímž dojde k přenosu zbraně do rukou hráče a následnému odstranění objektu ze scény.

Pro sběr nábojů se využívá obdobná logika. Když hráč vstoupí do kolizní oblasti s municí, systém zkontroluje, zda má pro daný typ nábojů (automatická puška, brokovnice, odstřelovací puška) dostatek volné kapacity. Pokud ano, náboje se přidají do odpovídající zásoby. V opačném případě zůstávají na místě a mohou být sebrány později, jakmile bude dostatek prostoru.

3.1.5 Krabice s vybavením

Pokud vám dojde munice u zbraně na zemi, nebo chcete změnit typ brnění, stačí dojít ke kterékoliv bíle označené krabici a stisknout klávesu E. Otevře se nové herní rozhraní, ve kterém můžete pomocí šipek vybrat, zda chcete používat brokovnici, automatickou nebo odstřelovací pušku. Stejně tak můžete zvolit typ brnění: lehké, střední nebo těžké. Střední brnění je neutrální, lehké vám sice zvýší rychlost pohybu, ale zároveň zpřístupní vyšší zranitelnost. U těžkého brnění je tomu naopak. Jakmile jste spokojeni s výběrem, klikněte na tlačítko „Apply“ a vraťte se zpět do boje!

3.1.6 Herní rozhraní (HUD)

V hlavním menu má hráč na výběr několik možností. Tlačítko „Play“ spustí samotnou hru. Volba „Controls“ otevře obrazovku, na které jsou vizuálně zobrazeny všechny ovládací klávesy. Možnost „Options“ poskytuje přístup k nastavení hlasitosti, kvality audia a vizuálních parametrů pro optimalizaci výkonu. Změny lze potvrdit tlačítkem „Apply“ nebo zrušit pomocí „Back“. Tlačítko „Quit Game“ slouží k ukončení hry.

Během hraní může hráč kdykoli otevřít pauzovací menu stiskem klávesy P. Toto menu obsahuje stejné volby jako hlavní menu, s výjimkou toho, že tlačítko „Play“ zde slouží k návratu zpět do hry a „Main Menu“ přesměrovává do úvodního menu. Pokud hráč během hry zemře, zobrazí se speciální nabídka se třemi možnostmi: „Restart“ (nové spuštění hry), „Quit to Main Menu“ (návrat do hlavního menu) a „Quit“ (vypnutí hry).

Hlavní herní HUD je komplexní a rozčleněný do několika oblastí. V levém dolním rohu se nacházejí tři informační lišty: horní zobrazuje stav baterie svítilny na zbrani, prostřední udává úroveň zdraví a spodní ukazuje energii postavy. Vedle těchto lišt se zobrazuje číslo aktuálního kola. Pravý dolní roh zobrazuje počet nábojů v zásobníku, počet náhradních nábojů a informace o aktuálně držené zbrani (její název a typ).

Střed obrazovky je vybaven zaměřovačem, jehož velikost se dynamicky mění v závislosti na rychlosti pohybu hráče, což napomáhá přesnějšímu míření. V horní části displeje je umístěn časovač, který odpočítává čas do začátku nového kola. V pravém horním rohu se nachází panel zobrazující změny atributů nepřátel, které vznikají na základě výstupu z neuronové sítě, a nad ním také počítadlo snímkové frekvence (FPS), které slouží ke sledování výkonnosti hry.

3.2 Autonomní agent

3.2.1 První verze agenta

Pro začátek jsem chtěl otestovat, jak probíhá trénování, vytváření modelu a nasazení natrénovaného modulu na simulaci v pluginu NevarokML. Proto jsem se pro první verzi rozhodl vytvořit jednoduchého agenta, který se dokáže pohybovat a postupně si v průběhu tréninku i simulace zlepšovat svou schopnost dosáhnout cíle.

Trénovací mapa

Na trénovací mapě se nachází hlavní entita typu NevarokMLTrainer, která slouží k tréninku agenta v rámci pluginu NevarokML. Pod ní funguje 270 podřízených entit typu NevarokMLEnvironment, z nichž každá obsahuje simulovaný herní svět pro konkrétního agenta. V každém prostředí je zároveň přítomna entita „agent“, na níž probíhá samotný trénink nebo simulace.

Dále se v každém prostředí nachází pouze podlaha (bez stěn — v této první verzi nejde o uzavřený prostor), pod kterou je umístěna spouštěcí oblast typu Trigger s názvem AbyssTrigger. Kromě toho je poblíž každého agenta ještě jeden Trigger s názvem GoalTrigger, který slouží jako cíl simulace.

Nevarok trenér

V NevarokMLTrainer se nejprve nastaví velikost tzv. „Action Space“ (akčního prostoru), aby NevarokMLEnvironment věděl, kolik typů akcí může agent provádět — buď jednu po druhé, nebo, pokud je to tak nastavené, i několik akcí během jednoho tiků tréninku či simulace. V první verzi, kdy agent zvládne pouze pohyb, má akční prostor pět akcí, které může provádět sekvenčně.

Dále se připravuje tzv. „Observation Space“ (pozorovací prostor), kam agent během hry ukládá určená data sloužící k učení a rozhodování v průběhu tréninku a simulace. Po vytvoření Action Space a Observation Space se aktivuje Blueprint uzel nazvaný PPO, což je algoritmus zvolený pro všechny tři verze tréninku a simulace. Tento uzel definuje klíčové hyperparametry:

- „Policy = MlpPolicy“ — agent používá plně propojenou vícervrstvou perceptronovou síť pro aproximaci politiky i hodnotové funkce.
- „Learning Rate“ — ovlivňuje velikost kroků při aktualizaci vah, nižší hodnota znamená stabilnější, ale pomalejší učení.
- „N Steps“ — počet kroků simulace sbíraných před aktualizací politiky.
- „Batch Size“ — počet přechodů (stav–akce–odměna) zpracovaných v jednom mini-batchi.

- „N Epochs“ — kolikrát se síť trénuje na nasbíraných datech před jejich zaházením.
- „Gamma“ — diskontní faktor pro budoucí odměny.
- „GAE Lambda“ — vyvažuje bias a variance při výpočtu výhodnosti pomocí Generalized Advantage Estimation.
- „Clip Range“ — omezuje změny politiky při aktualizaci, aby se zabránilo nestabilnímu učení.
- „Ent Coef“ — zvyšuje explorační odměňováním vyšší entropie politiky.
- „Vf Coef“ — určuje váhu hodnotové funkce v celkové ztrátě.
- „Max Grad Norm“ — slouží k ořezání gradientů a prevenci divergence.

Volitelné parametry zahrnují: „Use SDE“ pro stavový šum, „Sde Sample Freq“ pro jeho frekvenci, a „Verbose“ pro aktivaci logů během tréninku.

Po uzlu PPO následuje uzel Learn, který spouští samotný trénink či simulaci pomocí zvoleného algoritmu:

- „Device“ — umožňuje volbu mezi CPU a GPU.
- „Timesteps“ — celkový počet tréninkových kroků.
- „Eval Eps“ — počet evaluačních epizod.
- „Save Freq“ a „Log Interval“ — určují frekvenci ukládání modelů a zapisování logů.
- „Load Model Path“ a „Save Model Name“ — správa checkpointů a ukládání nových verzí.
- „Deterministic“ — zajišťuje deterministické chování při vyhodnocení po tréninku.

Volitelné ladicí volby jako „Show Tensorboard“, „Show Reward“, „Show Step Debug“ a „Show Reset Debug“ umožňují vizualizaci a sledování průběhu učení i chování agenta.

Nevarok prostředí

Po inicializaci prostředí se nejprve spustí událost „Event On Init“. Ta zkontroluje, zda je aktivní proměnná „Simulate“, která určuje, zda začít od začátku nebo pokračovat s uloženým modelem. Pokud je podmínka splněna, vytvoří se instance uzlu „NNEModule“, která načte a inicializuje uložený model. Tento uzel vyžaduje vstupy z „Action Sample“ a „Observation Sample“, aby model věděl, s jakými akčními a pozorovacími daty bude pracovat.

Následně se výstup modelu uloží do proměnné „Inference Model“, která slouží pro predikce akcí agenta. V této fázi se zároveň uloží počáteční pozice agenta do proměnné „Agent Initial Location“, získané z uzlu „Get Actor Location“. Tato pozice slouží k resetování prostředí na začátku každé epizody. Agent se pak přidá do seznamu „Ignore Actors“ pomocí uzlu „Add Unique“, aby byl ignorován při raycast kolizích během simulace.

Po inicializaci následuje průběžná simulace skrze událost „Event On Step“. Pokud je opět aktivní „Simulate“, zavolá se uzel „Predict“, který navrhne akce pro aktuální stav prostředí. Tyto akce se aplikují pomocí uzlu „Apply Actions“, zatímco aktuální stav se zaznamená přes uzel „Collect Observations“. Odměny za akce vyhodnotí funkce „Handle Rewards“, případně se zobrazí ladící výstup přes uzel „Debug“, který umožňuje sledování průběhu simulace.

Mezi hlavními ticky definovanými parametrem „Trainer->TickInterval“ lze volitelně spustit událost „Event On Step Skip“. Ta slouží například k aplikaci akcí v jemnějších krocích nebo k vyplnění mezer mezi hlavními kroky. Pozorování získaná během „Step Skip“ se do tréninku nezahrnují.

Nakonec se spustí událost „Event On Reset“, volaná před prvním krokem epizody a po jejím skončení. Slouží k resetování prostředí do výchozího stavu. Nejprve se aktivuje ladící uzel „Debug“ pro indikaci poslední odměny. Poté „Reset Triggers“ a „Reset Agent“ obnoví stav agentů a spouštěcích oblastí. Uzel „Clear“ vymaže nasbírané akce a pozorování („Action Sample“), aby se nová epizoda sbírala od nuly. Na závěr „Collect Observations“ uloží počáteční stav prostředí, který se použije jako výchozí bod pro trénink/simulaci.

Tento hlavní řetězec událostí zajišťuje kompletní cyklus tréninku a simulace agenta — od inicializace přes jednotlivé kroky s predikcemi modelu až po resetování prostředí na začátku nové epizody.

Při spuštění sběru pozorování („Collect Observations“) z hlavního cyklu se nejprve nastaví proměnná „Index Counter“ na výchozí hodnotu $(0, 0)$, která reprezentuje indexy pro iteraci senzorů v mřížce. Následně se tato proměnná postupně inkrementuje pomocí funkčních bloků „Index Plus Plus“, kde každé volání posune index o jeden krok ve směru osy Y (parametr „Step $dx = 0, dy = 1$ “). Tento mechanismus zajišťuje sekvenční průchod jednotlivými senzory nebo směry detekce.

Během každé iterace se načte aktuální pozice agenta i cílového senzoru. Pomocí uzlu „Get World Location“ se získá pozice senzoru, a pomocí „Get Actor Location“ se načte aktuální poloha agenta. Funkce „Find Look at Rotation“ vypočítá rotaci směrem k cílovému bodu, která je následně aplikována na senzor pomocí „Set World Rotation“. Tento postup zajistí, že senzor směřuje správným směrem před vlastním trasováním.

Následně proběhne vzorkování pomocí uzlu „Sample Box Line Trace“, který funguje jako raycast s definovanou délkou („Trace Length = 1000.0“) a kanálem („Trace Channel = Visibility“). Parametry „Difference Multiplier“ a „Velocity Multiplier“ škálují rozdíly vzdáleností a rychlostí pro ukládání do pozorovacího prostoru. Do uzlu jsou také připojeny seznamy „Ignore Actors“ a volitelně „Trace Tags“, které určují objekty, jež se mají ignorovat, a typy kolizí, které se vyhodnocují.

Výstupy raycastu se ukládají do „Observation Sample“, kde zahrnují indexy a hodnoty jako „Velocity Length Index“, „Difference Length Index“, „X/Y/Z Velocity Index“, „Direction Index“ a „Normal Index“. Tyto indexy reprezentují složky rychlosti, směru a normály zásahu, které slouží jako vstupní data pro neuronovou síť během tréninku nebo inferencí. Nepoužité výstupy se vedou na pin „Unused“.

Pro vizualizaci a ladění se využívají volitelné parametry na pravé straně uzlu: „Debug Trace“ pro zobrazení trasování paprsků ve scéně, „Debug Duration = 0.1“ pro určení délky zobrazení, „Debug Color“ pro barvu paprsku a „Debug Log“ pro textový výpis do konzole.

Celý tento postup zajišťuje, že agent nasbírání strukturovaná pozorování z okolního prostředí prostřednictvím senzorů a line trace, která se následně předávají do pozorovacího prostoru pro rozhodování a učení.

Po obdržení pokynu z hlavního řetězce se spouští událost „Apply Actions“, která převádí diskrétní akce agenta na pohyb ve světě. Nejprve se povolí pohybový tick pro komponentu „Nevarok ML Character Input“ prostřednictvím uzlu „Enable Movement Tick“. Tento krok zajistí, že budou zpracovávány vstupy pohybu v aktuálním kroku.

Následně se z „Action Sample“ získá hodnota diskrétní akce pomocí uzlu „Get Discrete Value“. Tato hodnota určuje, kterou akci agent provede. Hodnota je poté předána do uzlu „Add Discrete World Move Input“, který mapuje jednotlivé akce na konkrétní směry pohybu v herním světě:

- „Left (1)“ — pohyb vlevo,
- „Forward Left (-1)“ — diagonálně vlevo vpřed,
- „Forward (2)“ — vpřed,
- „Forward Right (-1)“ — diagonálně vpravo vpřed,
- „Right (3)“ — vpravo,
- „Backward Right (-1)“ — diagonálně vpravo vzad,
- „Backward (4)“ — vzad,
- „Backward Left (-1)“ — diagonálně vlevo vzad.

V tomto případě má povolen agent pouze pohyb vpřed, vzad či do stran, žádný diagonální pohyb.

Volba „Grounded Only“ omezuje aplikaci pohybu pouze na situace, kdy je agent v kontaktu se zemí. Po zpracování akce je pomocí uzlu „Force Movement Tick“ vynuceno provedení pohybového snímku, aby se změna polohy okamžitě projevila v simulaci.

Tento mechanismus umožňuje převést diskrétní rozhodnutí neuronové sítě na konkrétní pohyb agenta v prostředí a zajistit, že každá akce se projeví v rámci jednoho kroku simulace či tréninku.

Po obdržení pokynu z hlavního řetězce se spouští událost „Handle Rewards“, která vyhodnocuje odměny, penalizace a podmínky ukončení epizody agenta. Logika je rozdělena do několika částí, které se vyhodnocují sekvenčně.

První část představuje časový postih za každý krok („Penalty "Time/Step"“). Pomocí uzlu „Add Reward“ je agent penalizován malou zápornou hodnotou, typicky -0.01 , aby byl motivován dosáhnout cíle co nejrychleji a nezdržovat epizodu.

Následuje kontrola podmínek pro ukončení epizody. První z nich je dosažení maximálního počtu kroků („Done condition "Time out"“). Počet aktuálních kroků („Steps“) se porovná s „Max Steps“. Pokud je hodnota větší nebo rovna maxima, proměnná „Done“ se nastaví na pravdu a epizoda končí kvůli vypršení časového limitu.

Další podmínkou je pád agenta mimo herní prostor („Done condition "Fell out"“). Detekuje se pomocí uzlu „Is Triggered“, který je připojen k „Abyss Trigger“. V případě pádu epizoda končí a je přidělena penalizace „Penalty "Fell out"“ = -1.0 , aby se agent učil pádu vyhýbat.

Poslední vyhodnocovanou podmínkou je dosažení cíle („Done condition "Target reached"“). Uzel „Is Triggered“, připojený k „Goal Trigger“, detekuje vstup do cílové oblasti. Pokud je cíl dosažen, proměnná „Done“ se nastaví a agent obdrží kladnou odměnu „Bonus "Target reached"“ = $+1.0$, což motivuje opakování úspěšného chování.

Celá sekvence tedy kombinuje:

- drobnou penalizaci za zbytečné zdržování,
- výraznou penalizaci za neúspěch,
- vysokou odměnu za splnění cíle,

čímž vzniká základní reward shaping, který pomáhá neuronové síti efektivněji se učit správnému chování v prostředí.

Součástí logiky prostředí jsou také pomocné funkce, které zajišťují správné resetování a ladění epizod.

Funkce „Reset Triggers“ se stará o obnovu pozice cílového i propastného spouštěče na začátku každé epizody. Nejprve se vygeneruje náhodný vektor v rovině XY pomocí uzlu „XY Random Vector“, který je následně normalizován uzlem „Normalize“. Tento směr se vynásobí škálovacím faktorem (např. 500) a přičte se k aktuální pozici získané uzlem „Get Actor Location“. Výsledkem je nová pozice spouštěče, která se nastaví uzlem „Set Actor Location“. Poté se volají uzly „Reset Trigger“ pro cílový i propastný spouštěč („Goal Trigger“ a „Abyss Trigger“), čímž se zajistí jejich výchozí stav pro nadcházející epizodu.

Funkce „Reset Agent“ obnoví agenta do pozice uložené v proměnné „Agent Initial Location“. Tato pozice se aplikuje uzlem „Set Actor Location“ přímo na agenta a následně se pomocí uzlu „Reset Input“ vymažou všechny předchozí vstupy komponenty „Nevarok ML Character Input“. Tím je zajištěno, že agent vždy zahajuje každou epizodu z výchozího stavu bez zbytkového pohybu.

Funkce „Debug“ slouží k vizuálnímu i textovému ladění průběhu epizody. Využívá uzel „Draw Debug String“, který zobrazuje nad agentem text s aktuální kumulovanou odměnou („Episode Reward“). Barva textu a délka zobrazení („Duration“) jsou nastavitelné, což umožňuje snadné a rychlé sledování chování agenta během simulace nebo tréninku.

Společně tyto tři funkce — „Reset Triggers“, „Reset Agent“ a „Debug“ — zajišťují spolehlivé resetování prostředí, vynulování stavu agenta a intuitivní vizualizaci průběhu epizody.

Agent, GoalTrigger a AbyssTrigger

Tyto Actors neobsahují žádný vlastní kód — slouží pouze k zajištění provozuschopnosti systému NevarokMLEnvironment.

Agent disponuje několika sensory orientovanými do stran, aby získal informace o okolí. Dále obsahuje komponentu NevarokMLCharacterInput, která umožňuje jeho ovládání prostřednictvím NevarokMLEnvironment, rovněž tzv. obyčejný NevarokML senzor a senzor pro detekci zásahů.

AbyssTrigger obsahuje obyčejný NevarokML senzor a trigger typu NevarokMLBox-Trigger, který při kontaktu s agentem ukončí probíhající učicí epizodu, jak bylo výše popsáno. Goal Trigger funguje obdobně, liší se však ve své specifické funkci a využití v rámci prostředí.

Spolehlivost a vzhled modelu

Tabulka 3.1: Výsledky základního tréninku první verze agenta

Metrika	Hodnota
rollout/	
ep_len_mean	6.34
ep_rew_mean	0.937
time/	
fps	2502
iterations	20
time_elapsed	457
total_timesteps	1080000
train/	
approx_kl	0.0009790711
clip_fraction	0.01
clip_range	0.2
entropy_loss	-0.157
explained_variance	0.926
learning_rate	0.0003
loss	-0.00125
n_updates	3800
policy_gradient_loss	-0.00192
value_loss	0.00187

Výsledky základního tréninku první verze modelu zachycené v tabulce 3.1 ukazují statistiky dosažené po celkovém počtu 1080000 simulovaných kroků `total_timesteps` během 20 iterací. Průměrná délka epizody `ep_len_mean` činila 6.34 kroků a průměrná kumulovaná odměna na epizodu `ep_rew_mean` dosáhla hodnoty 0.937, což naznačuje, že agent se naučil úkol plnit s relativně vysokou úspěšností.

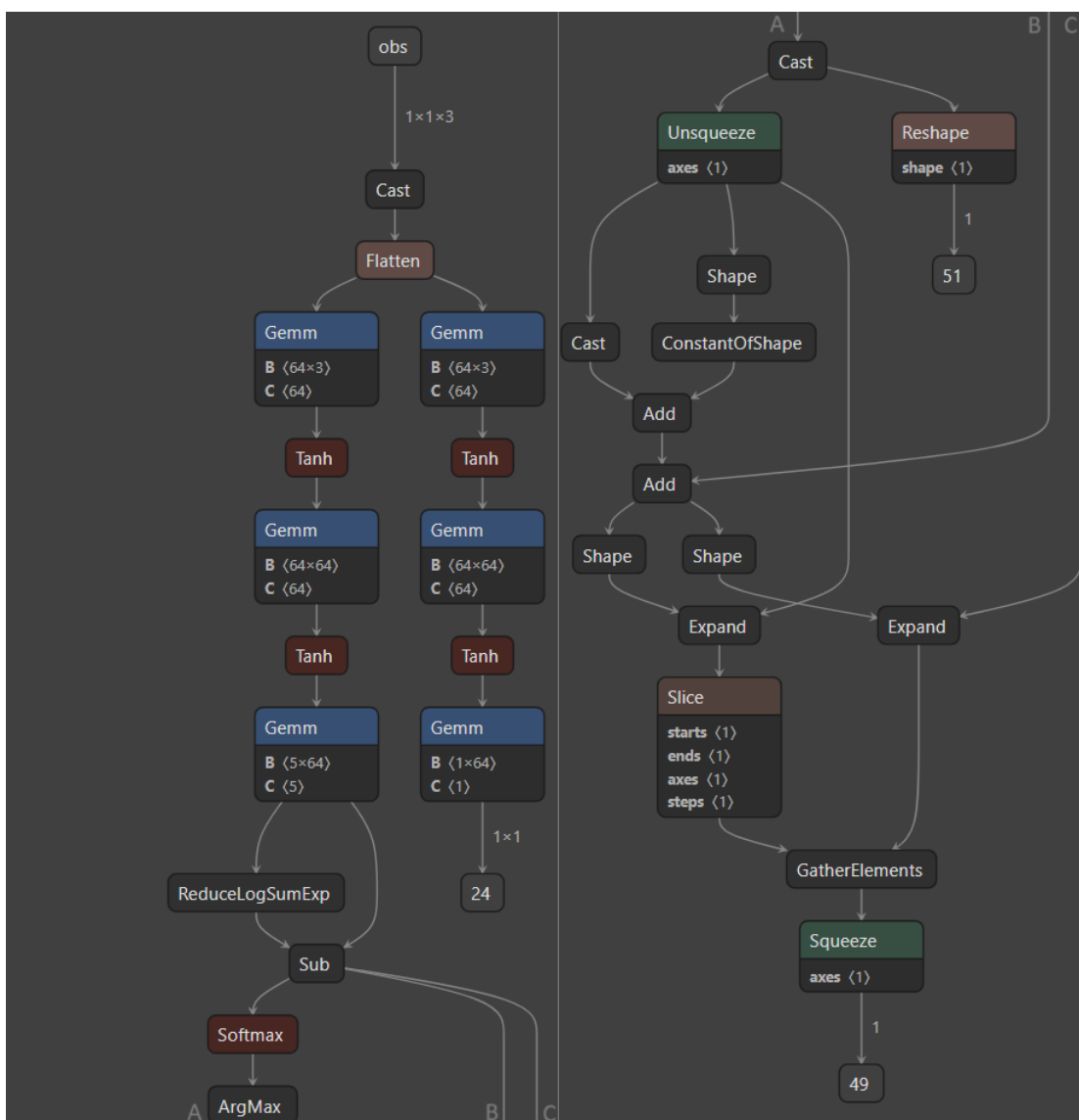
Trénink probíhal rychlostí přibližně 2500 snímků za sekundu (`fps`) a celkový čas trvání byl 719 sekund. Během tohoto období došlo k 3800 aktualizacím modelu `n_updates` při konstantní hodnotě `learning_rate` = 0.0003.

Hodnota přibližné KL divergence `approx_kl` je velmi nízká 0.00098, což naznačuje minimální odchylku nových politik od předchozích. Nízká hodnota `clip_fraction` = 0.01 v kombinaci s `clip_range` = 0.2 potvrzuje, že zásahy do gradientu byly vzácné.

Ztrátové funkce vykazují očekávané hodnoty: `policy_gradient_loss` = -0.00192, `value_loss` = 0.00187 a celková ztráta modelu `loss` = -0.00125. Hodnota `entropy_loss` = -0.157 odpovídá mírné míře explorační, která byla během tréninku mírně penalizována.

Důležitým ukazatelem je také `explained_variance` = 0.926, což znamená, že přibližně 92,6% variability cílových hodnot je vysvětleno predikcemi hodnotové funkce — potvrzuje to dobrý odhad stavu a stabilitu tréninku.

Funkčnost modelu je téměř stoprocentní, což odpovídá jeho zjednodušené povaze, vycházející z tréninku zaměřeného pouze na pohyb agenta směrem k cíli. Model se choval stejně přesně i při nasazení přímo na herní mapu, kde byl standardní pohyb agenta nahrazen tímto algoritmem. Tento model sloužil výhradně k ověření funkčnosti a osvojení si práce s pluginem NevarokML. Práce s tímto systémem je netradiční: netvoříte vlastní neurální vrstvy ani jejich funkce – ty zajišťuje samotný NevarokML. Místo toho definujete chování prostřednictvím funkcí jako CollectObservations, ApplyActions, HandleRewards apod., a NevarokML se následně pokusí model automaticky natrénovat a aplikovat.



Obrázek 3.1: Vzhled modelu z první verze agenta

Pro představu lze v obrázku 3.1 vidět exportovaný ONNX model první verze. Model reprezentuje jednoduchou neuronovou síť typu MLP se dvěma skrytými vrstvami a převážně actor-critic architekturou. Vstupní pole „obs“ má tvar $1 \times 1 \times 3$, které je následně převedeno uzlem „Cast“ a zploštěno (Flatten) na trojici hodnot představujících stavové pozorování.

Síť se dále větví do dvou částí:

1. Policy head (levá větev):

- obsahuje dvě plně propojené vrstvy („Gemm“ $3 \rightarrow 64$ a $64 \rightarrow 64$) s aktivizační funkcí „Tanh“, následované výstupní vrstvou ($64 \rightarrow 5$) pro pět diskretních akcí. Výstup je dále zpracován pomocí „ReduceLogSumExp“, normalizován funkcí „Softmax“ a převeden pomocí „ArgMax“, který vrací index akce s nejvyšší pravděpodobností.

2. Value head (pravá větev):

- má stejnou strukturu skrytých vrstev ($3 \rightarrow 64$ a $64 \rightarrow 64$ s „Tanh“) a končí výstupní vrstvou ($64 \rightarrow 1$), která poskytuje skalární odhad hodnoty stavu (baseline pro algoritmus PPO).

V pravé části grafu jsou také podpůrné uzly „Unsqueeze“, „Shape“, „Slice“, „GatherElements“, které slouží k úpravě tvarů tenzorů a indexování výstupů při inferenci napříč více instancemi prostředí.

3.2.2 Druhá verze agenta

Druhá verze agenta již obsahuje kompletní logiku chování – dokáže se pohybovat, otáčet a nakonec i střílet na nepřítele.

Změny v trénovací mapě

Trénovací mapa se do určité míry změnila. Místo podlahy bez stěn má nyní každý NevarokMLEnvironment podlahu se stěnami, aby agent během základního tréninku nemohl spadnout mimo vymezenou oblast. Zároveň se tato oblast zdvojnásobila. Agent na mapě zůstal beze změn, avšak obě původní spouštěcí oblasti GoalTrigger a AbyssTrigger byly odstraněny. Místo nich se nyní na mapě nachází nový Actor s názvem Enemy (nepřítel).

Změny v trenérovi

V NevarokMLTrainer byly provedeny drobné úpravy. Trénér nyní nastavuje velikost „Action Space“ nikoliv pomocí jediné hodnoty, ale přijímá pole o třech prvcích: první s rozsahem 0–8, druhý 0–2 a třetí 0–1. V pozadí tak NevarokML vytváří akční krychli, která umožňuje agentovi provádět až tři akce současně namísto sekvenčního provádění. Celkový počet možných kombinací akcí se díky tomu zvýšil z 5 na 54.

Byl také rozšířen „Observation Space“ — agent nyní sbírá o dvě další hodnoty navíc, což bylo nutné kvůli novým typům akcí.

Navíc bylo přidáno ukládání počáteční rotace agenta, která se využívá při resetu a orientaci v každé nové epizodě.

Změny v prostředí

Základní inicializační část NevarokMLEnvironment zůstala beze změny, avšak došlo ke změnám v ostatních částech.

Ve funkci `CollectObservations` byla přidána část logiky, která zajišťuje výpočet vzdálenosti a úhlové odchylky (yaw rozdílu) mezi agentem a nepřítelem. Tyto hodnoty se následně zapisují do pozorovacího prostoru.

Nejprve se pomocí uzlu „Sample Box Line Trace“ detekuje vzdálenost k objektu před agentem. Výstup „Difference Length“ se uloží do proměnné „Distance To Enemy“, a poté se umístí do pozorovacího pole pomocí uzlu „Set Box Value“.

Současně probíhá výpočet směrového rozdílu mezi agentem a cílem. Uzel „Get Actor Rotation“ spolu s „Break Rotator“ extrahují hodnotu yaw agenta, zatímco funkce „Find Look at Rotation“ vypočítá požadovaný úhel směru k cíli. Tyto hodnoty se ukládají jako „Agent Yaw“ a „Look at Yaw“.

Poté se úhlový rozdíl zpracuje přes uzel „Normalize Axis“, což zajišťuje cyklickou konzistenci výsledku v rozsahu $[-180^\circ, 180^\circ]$. Odchylka se převede na absolutní hodnotu a uloží jako „Yaw Diff“, opět pomocí uzlu „Set Box Value“.

Pro každé zápis do pozorovacího pole se využívá inkrementační čítač „Index Counter“, který pomocí funkce „Index Plus Plus“ zajišťuje, že jednotlivé hodnoty se ukládají do oddělených indexů bez přepisování.

Tato sekvence poskytuje agentovi nové prostorové informace užitečné pro navigaci — konkrétně vzdálenost k cíli (nepříteli) a velikost směrové odchylky, kterou je nutné napravit pro správné natočení k cílové oblasti.

Funkce pro aplikaci akcí nyní zpracovává výstup politiky agenta rozdělený do tří větví pomocí „Multi Discrete“ reprezentace: pohyb, rotace a střelba.

Pomocí uzlu „Sequence“ je zajištěno sekvenční provedení všech akčních větví:

1. Pohyb:

- První větev zpracovává výstup s indexem 0 pomocí uzlu „Get Multi Discrete Value“, a výsledek je předán do uzlu „Add Discrete World Move Input“. Tento uzel interpretuje hodnoty 0–8 jako pohyb vpřed, vzad, do stran a nově také diagonálně.

2. Rotace:

- Druhá větev (index 1) určuje otáčení — hodnoty 1 a 2 představují rotaci vlevo či vpravo, realizovanou pomocí uzlu „Add Discrete Turn Input“.

3. Střelba:

- Třetí větev (index 2) aktivuje střelbu pouze při splnění dvou podmínek: agent je dostatečně blízko cíle a zároveň je správně natočen. Pokud jsou obě podmínky splněny, vyvolá se uzel „Trigger Fire Action“, který realizuje logiku střelby.

Každá akce je zprostředkována komponentou „Nevarok ML Character Input“. Nejprve je aktivován pohybový tick pomocí „Enable Movement Tick“, a po dokončení celé sekvence následuje „Force Movement Tick“, který zajišťuje okamžité aplikování akcí během jednoho simulačního kroku.

Funkce **Trigger Fire Action** představuje zjednodušený systém střelby, který je spuštěn po uplynutí 0,5s díky uzlu **Do Once** a opakovanému obnovení proměnné **Can Fire** přes **Set Timer by Function Name**. Kód nejprve ověří, zda je střelba povolena (**Can Fire == true**); pokud ano, pokračuje dále.

Ze scény se získá pozice a směr kamery pomocí uzlů **Get World Location** a **Get World Rotation**, a vektor směru se prodlouží o určitou vzdálenost operací **+**. Tento výsledný směr provede **Line Trace by Channel**, který simuluje paprsek vpřed ve směru pohledu agenta a ignoruje samotného agenta.

Výsledek raycastu se zpracuje přes **Break Hit Result**. Pokud dojde ke kolizi (**Blocking Hit == true**) a objekt odpovídá nepříteli (**Hit Actor == Enemy**), agent získá odměnu **+2,0** a spustí se funkce **Apply Damage** s hodnotou **10**. Pokud je zásah mimo nepřítele nebo není žádný hit, agent dostane penalizaci **-0,2**.

Na závěr se proměnná **Can Fire** resetuje na **false**, a opět se spustí časovač s hodnotou **0,5s**. Po jeho uplynutí se střelba opět povolí.

V novém systému odměn je agent průběžně motivován k dosažení cíle, správné orientaci, optimální vzdálenosti od cíle a eliminaci nepřítele, přičemž je penalizován za chyby, neaktivitu nebo vypršení časového limitu.

1. Odměna za správné natočení:

- Pokud je rozdíl směru („YawDiff“) mezi agentem a směrem k nepříteli malý, agent získá odměnu až do **0,03** — čím přesněji míří, tím vyšší odměna. Při větším odchýlení ztrácí až **-0,05**.

2. Odměna za rotaci během pohybu:

- Agent je odměněn pouze tehdy, pokud se pohybuje vpřed a současně otáčí směrem k cíli, což se detekuje pomocí skalárního součinu forward vektoru a směru k cíli. Odměna je škálována až do **0,03**.

3. Odměna za ideální vzdálenost:

- Agent získává mírnou odměnu (maximálně **0,03**) za udržování optimální vzdálenosti od cíle. Pokud je příliš blízko nebo příliš daleko, je penalizován hodnotou **-0,03**.

4. Eliminace nepřítele:

- Pokud agent úspěšně eliminuje nepřítele, epizoda se ukončí a je přidělena hlavní odměna o hodnotě **5,0**.

5. Penalizace za timeout:

- Pokud agent epizodu nevyřeší a dojde k dosažení maximálního počtu kroků, obdrží výrazně negativní odměnu $-10,0$, což slouží jako jasný signál selhání.

Funkce „Reset Triggers“ byla nahrazena funkcí „Reset Enemy“, která funguje obdobně, avšak navíc nastavuje poškozenému nepříteli proměnnou „Death“ na false a obnovuje jeho život na hodnotu 50.

Funkce „Reset Agent“ se kromě nápravy počáteční rotace agenta nezměnila.

Agent samotný nebyl nijak rozšířen, nicméně byl vytvořen nový actor „Enemy“, jenž obsahuje základní NevarokML senzor a také kód zpracovávající poškození a případný konec života nepřítele. Při zásahu se spustí událost **Event AnyDamage**, která volá vlastní událost **Health Modifier** s předanou hodnotou poškození. Ta nejdříve odečte hodnotu **Damage** od aktuální proměnné **Health** a poté výsledek pomocí funkce **Clamp (Float)** omezí na rozsah $[0,0]$ až $[\text{Max Health}]$, aby nedošlo k přetečení nebo záporné hodnotě. Upravená hodnota se uloží zpět do proměnné **Health**.

Následuje kontrola, zda je zdraví menší nebo rovno nule. Pokud ano, aktivuje se větev **True**, která vyvolá uzel **Do Once** a poté vlastní událost **Dead**. Tím se proměnná **Death** nastaví na true, což signalizuje zabití nepřítele. Při vyhodnocení odměn to automaticky spustí reset prostředí.

Spolehlivost modelu

Tabulka 3.2: Výsledky základního tréninku druhé verze agenta

Metrika	Hodnota
rollout/	
ep_len_mean	36.4
ep_rew_mean	13.8
time/	
fps	2502
iterations	59
time_elapsed	1273
total_timesteps	3240000
train/	
approx_kl	0.007343081
clip_fraction	0.0368
clip_range	0.2
entropy_loss	-0.968
explained_variance	0.499
learning_rate	0.0003
loss	0.501
n_updates	11600
policy_gradient_loss	-0.00472
value_loss	1.02

Jak lze vidět z tabulky 3.2, výsledky u první plně implementované verze algoritmu se výrazně liší. Jednotlivé epizody trvaly déle než u verze první, protože agent musel nejen dosáhnout cíle chůzí, ale také najít nepřítele, otočit se na něj, zahájit útok a eliminovat ho. Délka učení se musela přibližně ztrojnásobit, aby se dosáhlo spolehlivého chování agenta.

Ztráta podle policy gradientu se zvětšila z $-0,00192$ na $-0,00472$, což ukazuje na silnější gradientové signály a výraznější změny v politice agenta. Přibližná KL divergence vzrostla z $0,00098$ na $0,00734$, což znamená, že se politika během tréninku aktualizovala agresivněji — typický jev při postupném zpřesňování chování agenta.

Entropy loss klesla z $-0,157$ až na $-0,968$, čímž došlo k výraznému snížení náhodnosti v rozhodování — agentovo chování se stalo více deterministické, což obvykle svědčí o pozitivní konvergenci. Nicméně explained variance poklesla z $0,926$ na $0,499$, což papírově znamená, že přesnost hodnotové funkce (kritiky) se zhoršila a predikce budoucích odměn jsou méně spolehlivé. Tento trade-off je v hlubokém posilovaném učení běžný, zejména když agent začíná používat konkrétní strategie.

Jedním z pozoruhodných rozdílů je také nárůst value loss z $0,00187$ na $1,02$, což může indikovat určitou nestabilitu v hodnotové funkci během tréninku.

Bohužel tento model vyžaduje úpravy. Ignoruje některá pravidla odměňování: místo udržování optimální vzdálenosti od nepřítele se přiblíží co nejvíce, aby maximalizoval šanci na úspěšnou střelu — protože odměny za zásah jsou výrazně vyšší než penalizace za nesprávnou vzdálenost. Navíc chybí implementovat poslední funkcionalitu.

3.2.3 Finální verze agenta

Modifikace prostředí

Jednoduchá střelecká funkce byla nahrazena propracovanější logikou umístěnou přímo na zbraních vytvářených pro agenta. Odměna za zásah byla zároveň snížena. Výsledkem je funkce, která je vizuálně i logikou mnohem vyspělejší. Navíc je možné vybírat mezi třemi typy zbraní, což přináší další novou funkcionalitu.

Agent si před začátkem každého kola dokáže zapamatovat, jakou zbraň a brnění na konci předchozího kola používal hráč, a podle toho zvolí zbraň i brnění, které mu nejlépe pomohou hráče porazit.

Ve funkci odměn bylo navíc přidáno jedno nové omezení a jedno původní omezení upraveno:

1. Kombinovaná odměna:

- Pokud jsou současně splněny podmínky správného natočení a správné vzdálenosti od cíle, agent získá odměnu ve výši $+0,1$. V opačném případě je penalizován hodnotou $-0,1$.

2. Eliminace nepřítele:

- Pokud agent úspěšně zneškodní nepřítele, epizoda se ukončí a je přidělena hlavní odměna. Výchozí hodnota činí $5,0$, ale je násobena koeficientem závislým na vzdálenosti v okamžiku zásahu. Nejvyšší odměna je udělena při optimálním odstupu od cíle.

Spolehlivost a vzhled finálního modelu

Tabulka 3.3: Výsledky základního tréninku finální verze agenta

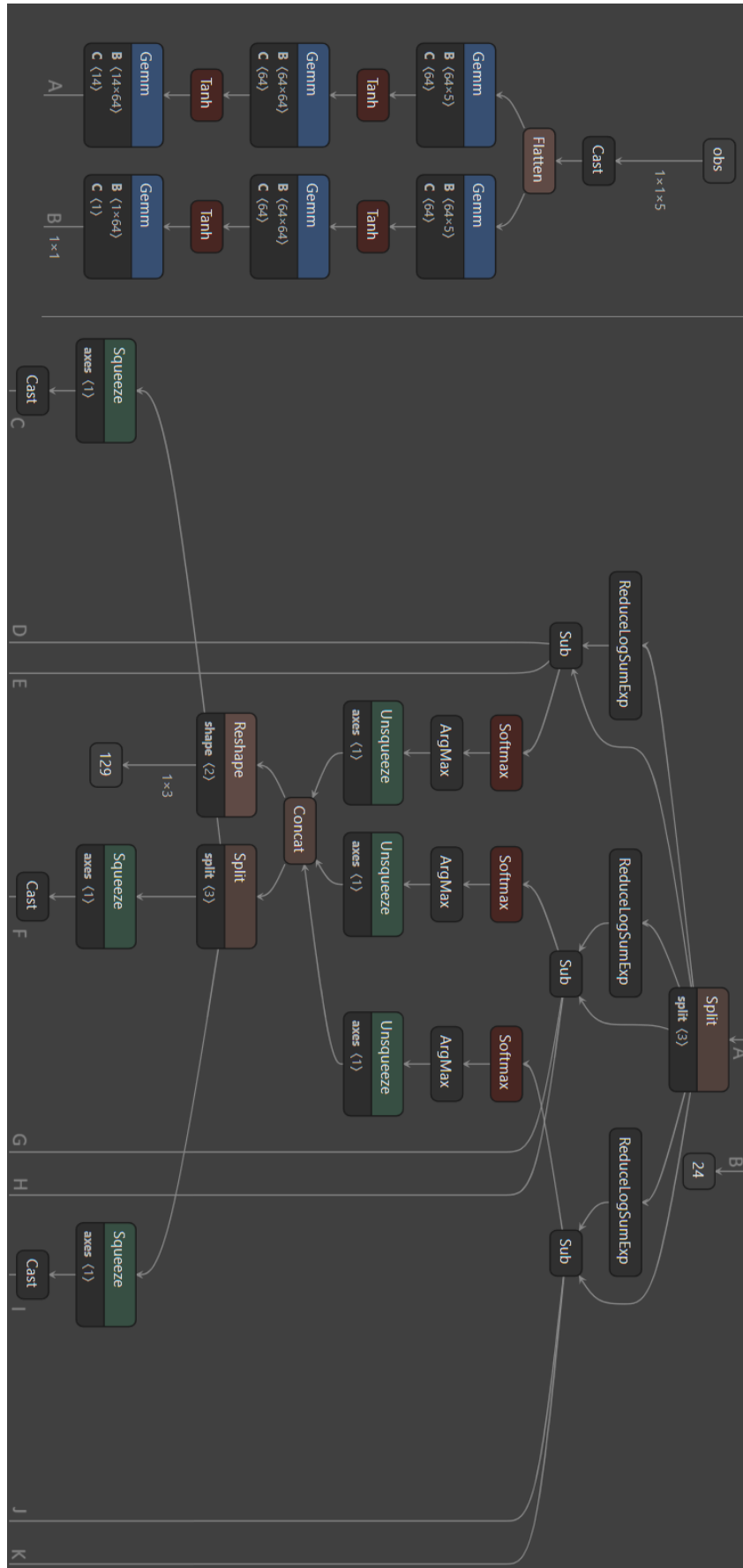
Metrika	Hodnota
rollout/	
ep_len_mean	27.4
ep_rew_mean	8.8
time/	
fps	2500
iterations	59
time_elapsed	1237
total_timesteps	3240000
train/	
approx_kl	0.004143081
clip_fraction	0.0332
clip_range	0.2
entropy_loss	-0.994
explained_variance	0.604
learning_rate	0.0003
loss	0.301
n_updates	11600
policy_gradient_loss	-0.00432
value_loss	0.7

Tabulka 3.3 ukazuje výsledky finální verze modelu, která v porovnání s verzí druhou vykazuje zlepšení ve všech sledovaných oblastech. Jednotlivé epizody trvaly mírně kratší dobu než u předchozí verze — agent nyní dokáže rychleji najít nepřítele, otočit se k němu, zahájit útok a eliminovat ho, přičemž tréninková doba zůstala zachována.

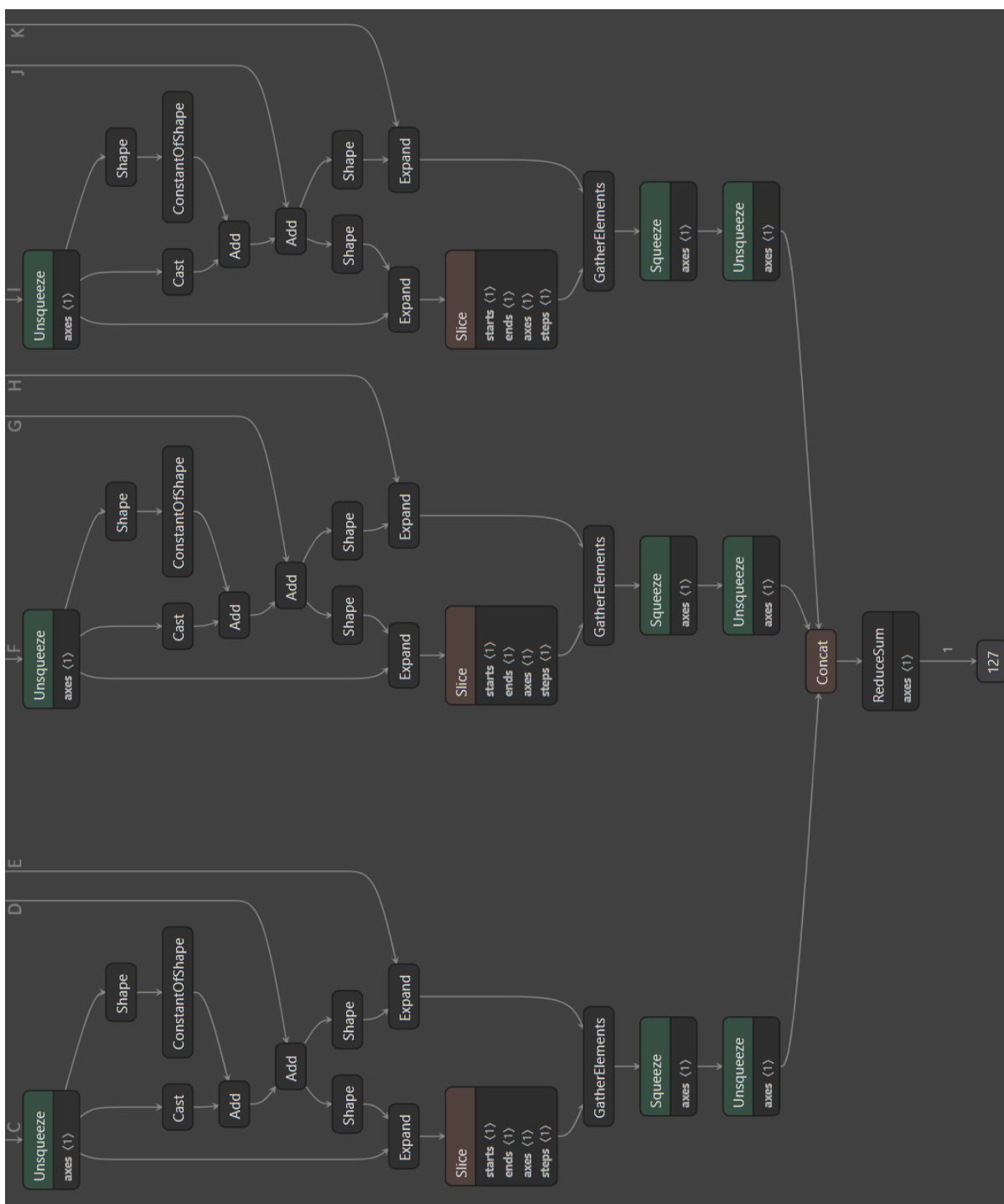
„Ztráta podle policy gradientu“ se snížila z $-0,00472$ na $-0,00432$, což poukazuje na méně výrazné gradientové změny v agentově politice — žádoucí jev. „Přibližná KL divergence“ klesla z $0,00734$ na $0,00414$, což znamená, že aktualizace politiky během tréninku probíhaly méně agresivně — agentovo chování je nyní stabilnější a přesnější. „Entropy loss“ se ještě více snížila z $-0,968$ na $-0,994$, což odráží další pokles náhodnosti v rozhodování. „Explained variance“ se zlepšila z $0,499$ na $0,604$, což znamená lepší odhad hodnoty stavu a spolehlivější predikce. „Value loss“ klesla z $1,02$ na $0,70$, což naznačuje menší nestabilitu hodnotové funkce.

Finální agent dokáže správně vybrat zbraň a brnění, pronásledovat hráče, natočit se na něj a začít střílet. Přesto není systém bez nedostatků: při pohybu hráče občas agent místo jemné úpravy rotace udělá celkový obrat, než správně zamíří.

Obrázky 3.2 a 3.3 ukazují vizuální podobu finálního modelu.



Obrázek 3.2: Vzhled modelu z finální verze agenta 1/2



Obrázek 3.3: Vzhled modelu z finální verze agenta 2/2

3.2.4 Shrnutí všech verzí

První verze agenta se sice učí a funguje nejlépe, ale zároveň je ze všech verzí nej-jednodušší, protože zvládá pouze pohyb. Hráč proto tento model dokáže snadno porazit. Druhá verze sice zahrnuje veškerou logiku, ale její natrénování se nedařilo — agent se neustále těsně držel nepřítele, což sice mohlo vést k vítězství, ale působilo nepřirozeně. Finální verze je ze všech nejvyspělejší — učí se a funguje téměř bezchybně: skutečně pronásleduje hráče, mění výbavu, útočí apod. Tato verze je zároveň nejobtížnější na porážku, což byl zamýšlený cíl. Bohužel, přestože se agent během hry dále učí, ve hře je výrazně méně agentů než na trénovací mapě, takže trénink během hraní je téměř zanedbatelný.

Závěr

Hlavním cílem této diplomové práce bylo navrhnout jednoduchého autonomního agenta, který dokáže za pomoci strojového učení a změny vybavení porazit protihráče.

Nejprve jsem popsal existující AI agenty ve hrách a nejčastěji používané techniky a algoritmy. Mezi zmíněnými agenty byli AlphaStar, OpenAI Five a Voyager; z algoritmů a technik pak PPO, Deep Q-Network, Self-Play a Population-Based Training.

V další části jsem uvedl technologie použité při tvorbě samotné hry a agenta — např. Unreal Engine či plugin NevarokML. Popsán byl také návrh hry i samotného agenta.

Závěrečná část práce se věnovala implementaci jednotlivých modulů hry, jako jsou hráčská postava, zbraně, systém kol, získávání vybavení (zbraní a brnění) a především implementaci, trénování a aplikaci agenta. Tento agent využívá logiku vytvořenou pomocí strojového učení a je schopen pohybu, rotace, útoku i výměny výbavy.

První verze agenta zvládala pouze pohyb, přičemž jejím hlavním účelem bylo osvojení si práce s pluginem NevarokML. Druhá verze, která byla první plně funkční, fungovala pouze částečně — agent se občas zcela zasekl a nevěděl, jak jednat, nebo se příliš přibližoval k nepříteli místo toho, aby si udržoval odstup. Finální verze agenta, navzdory několika drobným chybám, prokázala schopnost samostatného učení a fungování v prostředí. Dokáže se pohybovat, otáčet, útočit na hráče a po každém kole si správně změnit výbavu.

Literatura

- [1] GOOGLE. *AlphaGo*. Online. GOOGLE. Google DeepMind. B. r. Dostupné z: <https://deepmind.google/technologies/alphago/>. [cit. 2025-05-10].
- [2] CONTRIBUTORS, Wikipedia. *ChatGPT*. Online. In: Wikipedia: the free encyclopedia. San Francisco (CA): Wikimedia Foundation, 2001-, https://en.wikipedia.org/wiki/Main_Page. Dostupné z: <https://en.wikipedia.org/w/index.php?title=ChatGPT&oldid=1290307244>. [cit. 2025-05-10].
- [3] CONTRIBUTORS, Wikipedia. *Dota 2*. Online. In: Wikipedia: the free encyclopedia. San Francisco (CA): Wikimedia Foundation, 2001-. Dostupné z: https://en.wikipedia.org/w/index.php?title=Dota_2&oldid=1288870722. [cit. 2025-05-10].
- [4] JADERBERG, Max; DALIBARD, Valentin a OSINDERO, Simon et al. *Population Based Training of Neural Networks*. Online, arxiv. USA: Cornell University, 2017. Dostupné z: <https://arxiv.org/abs/1711.09846>. [cit. 2025-05-10].
- [5] *MuJoCo*. Online. C2021. Dostupné z: <https://mujoco.org/>. [cit. 2025-05-10].
- [6] TEAM, OpenAI Five. *OpenAI Five defeats Dota 2 world champions*. Online. OpenAI. 2019. Dostupné z: <https://openai.com/index/openai-five-defeats-dota-2-world-champions/#timeline>. [cit. 2025-05-10].
- [7] *Proximal Policy Optimization*. Online. OpenAI. 2017. Dostupné z: <https://openai.com/index/openai-baselines-ppo/>. [cit. 2025-05-10].
- [8] SCHULMAN, John; LEVINE, Sergey; MORITZ, Philipp; JORDAN, Michael I. a ABBEEL, Pieter. *Trust Region Policy Optimization*. Online, arxiv. USA: Cornell University, 2015. Dostupné z: <http://arxiv.org/abs/1502.05477>. [cit. 2025-05-10].
- [9] SCHULMAN, John; WOLSKI, Filip; DHARIWAL, Prafulla; RADFORD, Alec a KLIMOV, Oleg. *Proximal Policy Optimization Algorithms*. Online, arxiv. USA: Cornell University, 2017. Dostupné z: <https://arxiv.org/abs/1707.06347>. [cit. 2025-05-10].
- [10] SCHULMAN, John; MORITZ, Philipp; LEVINE, Sergey; JORDAN, Michael a ABBEEL, Pieter. *High-Dimensional Continuous Control Using Generalized*

- Advantage Estimation*. Online, arxiv. USA: Cornell University, 2018. Dostupné z: <https://arxiv.org/abs/1506.02438>. [cit. 2025-05-10].
- [11] VINYALS, Oriol; BABUSCHKIN, Igor; CHUNG, Junyoung et al. *AlphaStar: Mastering the real-time strategy game StarCraft II*. Online. Google DeepMind. B. r. Dostupné z: <https://deepmind.google/discover/blog/alphastar-mastering-the-real-time-strategy-game-starcraft-ii/>. [cit. 2025-05-10].
- [12] WANG, Guanzhi; XIE, Yuqi a JIANG, Yunfan et al. *Voyager: An Open-Ended Embodied Agent with Large Language Models*. Online. MineDojo. B. r. Dostupné z: <https://voyager.minedojo.org/>. [cit. 2025-05-10].
- [13] WATKINS, Christopher J. C. H. a DAYAN, Peter. Q-learning. Online. *Machine Learning*. 1992, roč. 8, č. 11, s. 279—292. Dostupné z: <https://doi.org/10.1007/BF00992698>. [cit. 2025-05-10].
- [14] CONTRIBUTORS, Wikipedia. *Deep Blue (chess computer)*. Online. In: Wikipedia: the free encyclopedia. San Francisco (CA): Wikimedia Foundation, 2001-. Dostupné z: [https://en.wikipedia.org/w/index.php?title=Deep_Blue_\(chess_computer\)&oldid=1288103584](https://en.wikipedia.org/w/index.php?title=Deep_Blue_(chess_computer)&oldid=1288103584). [cit. 2025-05-10].
- [15] CONTRIBUTORS, Wikipedia. *StarCraft II*. Online. In: Wikipedia: the free encyclopedia. San Francisco (CA): Wikimedia Foundation, 2001-. Dostupné z: https://en.wikipedia.org/w/index.php?title=StarCraft_II&oldid=1286300229. [cit. 2025-05-10].
- [16] ZHANG, Ruize; XU, Zelai a MA, Chengdong et al. *A Survey on Self-play Methods in Reinforcement Learning*. Online, arxiv. USA: Cornell University, 2024. Dostupné z: <https://arxiv.org/abs/2408.01072>. [cit. 2025-05-10].
- [17] *Unreal Engine* [online]. USA: Epic Games, 2025 [cit. 2025-07-31]. Dostupné z: www.unrealengine.com
- [18] *NevarokML* [online]. China: Kyrylo Mishakin, 2025 [cit. 2025-07-31]. Dostupné z: <https://nevarok.com/nevarok-ml/>
- [19] *ONNX* [online]. USA: Microsoft, AMD et., 2025 [cit. 2025-07-31]. Dostupné z: www.onnx.ai
- [20] *Neural Network Engine* [online]. USA: Epic Games, 2025 [cit. 2025-07-31]. Dostupné z: <https://dev.epicgames.com/documentation/en-us/unreal-engine/neural-network-engine-overview-in-unreal-engine>
- [21] SUTTON, Richard S. a BARTO, Andrew G. *Reinforcement learning: an introduction*. Second edition. Adaptive computation and machine learning. Cambridge, Massachusetts: The MIT Press, [2018]. ISBN 978-0-262-03924-6.
- [22] MNIH, Volodymyr; KAVUKCUOGLU, Koray; SILVER, David et al. *Playing Atari with Deep Reinforcement Learning*. Online, arxiv. USA: Cornell University, 2013. Dostupné z: <https://arxiv.org/abs/1312.5602>. [cit. 2025-07-31].

- [23] MNIH, Volodymyr; KAVUKCUOGLU, Koray; SILVER, David et al. Human-level control through deep reinforcement learning. Online. *Nature*. 2015, č. 518, s. 529—533. Dostupné z: <https://doi.org/10.1038/nature14236>. [cit. 2025-07-31].
- [24] TEAM, AlphaStar. *AlphaStar: Mastering the real-time strategy game StarCraft II*. 2019. Dostupné z: <https://deepmind.google/discover/blog/alphastar-mastering-the-real-time-strategy-game-starcraft-ii/>. [cit. 2025-07-31].
- [25] TEAM, OpenAI Five. *OpenAI Five*. 2018. Dostupné z: <https://openai.com/index/openai-five/>. [cit. 2025-07-31].
- [26] WANG, Guanzhi; XIE, Yuqi a JIANG, Yunfan et al. *Voyager: An Open-Ended Embodied Agent with Large Language Models*. 2023. Dostupné z: <https://voyager.minedojo.org/>. [cit. 2025-07-31].
- [27] PATHMAKUMAR, Thejus et al. *A Reinforcement Learning Based Dirt-Exploration for Cleaning-Auditing Robot*. 2021. Dostupné z: https://www.researchgate.net/figure/The-reinforcement-learning-architecture-based-on-Proximal-Policy-Optimization-fig3_357019705. [cit. 2025-07-31].
- [28] STROUSE, DJ et al. *Collaborating with Humans without Human Data*. Online, arxiv. USA: Cornell University, 2021. Dostupné z: <https://arxiv.org/abs/2110.08176>. [cit. 2025-07-31].
- [29] JADERBERG, Max. *Population based training of neural networks*. 2017. Dostupné z: <https://deepmind.google/discover/blog/population-based-training-of-neural-networks/>. [cit. 2025-07-31].
- [30] ELSAYED, Medhat et al. *AI-enabled Future Wireless Networks: Challenges, Opportunities and Open Issues*. 2021. Dostupné z: https://www.researchgate.net/publication/349913716_AI-enabled_Future_Wireless_Networks_Challenges_Opportunities_and_Open_Issues. [cit. 2025-07-31].

Příloha A

Jak zprovoznit program

1. Nejprve si stáhněte soubor přiložený k diplomové práci, který obsahuje samotný projekt a také pluginy, jež je nutné přidat do Unreal Engine.
2. Pokud nemáte nainstalovaný „Epic Games Store“, stáhněte si jej ze stránky <https://store.epicgames.com/>. V pravém horním rohu klikněte na tlačítko „Download“ a spusťte stažený instalátor, čímž nainstalujete „Epic Games Launcher“.
3. Po spuštění aplikace si vytvořte účet (pokud jej ještě nemáte) buď přímo v „Epic Games Store“, nebo prostřednictvím „Epic Games Launcher“, a přihlaste se.
4. Po přihlášení klikněte v levém menu na položku „Unreal Engine“.
5. V horní části aplikace se zobrazí nové možnosti — vyberte položku „Library“.
6. V sekci „Engine Versions“ klikněte na tlačítko „+“. Objeví se nabídka výběru verze engine. Zvolte verzi 5.2.1 a spusťte instalaci. Umístění instalace můžete ponechat výchozí, ale zapamatujte si ho — bude potřeba později!
7. Po dokončení instalace přejděte do složky s nainstalovaným engine, která se bude jmenovat např. `UE_5.2`. Do této složky vložte podsložku **Engine** ze složky „Plugins“, která je umístěna vedle složky projektu „RoboHell“. Tímto krokem zajistíte správné nainstalování potřebných pluginů.
8. Projekt spustíte dvojklikem na soubor „RoboHell.uproject“ ve složce projektu. První spuštění může trvat déle, protože probíhá příprava engine a kompilace projektu.
9. Po načtení projektu jej zatím nespouštějte pomocí zeleného trojúhelníku! Před prvním spuštěním je třeba provést ještě několik kroků.
10. V levém horním rohu klikněte na „Edit“ a v rozbaleném menu vyberte „Editor Preferences“.
11. V zobrazeném nastavení sjedźte v levém panelu dolů k sekci „Plugins“ a rozklikněte „NevarokML“.

12. Klikněte na tlačítko „Refresh Details“. Vizualně se nic nezmění, ale tímto se plugin správně načte v editoru.
13. Tímto by mělo být vše připraveno. Projekt by nyní měl být plně funkční. Doporučuji však přečíst i kapitolu „Jak opravit problém s projektem“, která řeší častou chybu při spouštění.

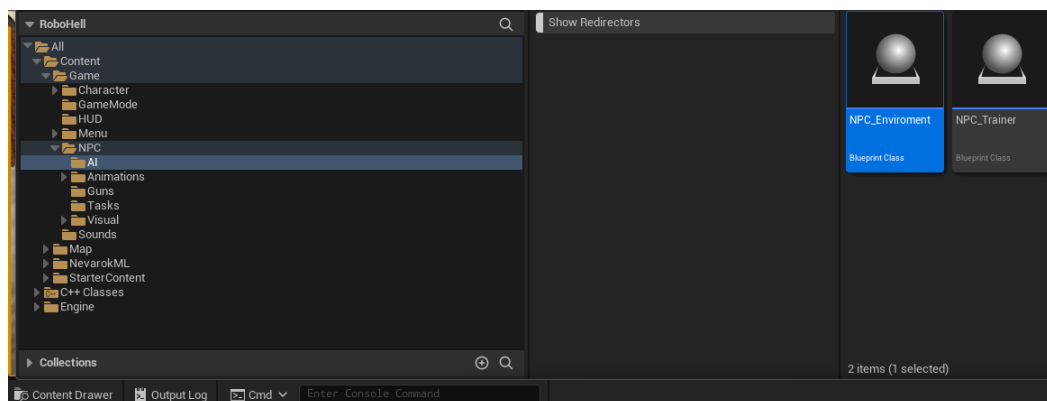
Příloha B

Jak opravit problém s programem

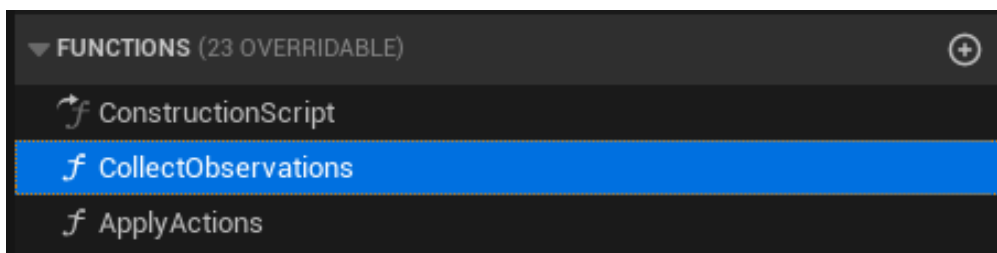
1. Pokud se vám zdá, že agent za vámi nechodí a odchází jedním směrem pryč, nebo se na obrazovce zobrazí chybová hláška zobrazená na obrázku 1, tato příloha vám poradí, jak danou chybu opravit.

```
[UNevarokMLSample::ValidateBoxValue] x(-5) < 0 || x(-5) >= Space->Shape[0](1)
[UNevarokMLSample::ValidateBoxValue] x(-4) < 0 || x(-4) >= Space->Shape[0](1)
[UNevarokMLSample::ValidateBoxValue] x(-5) < 0 || x(-5) >= Space->Shape[0](1)
[UNevarokMLSample::ValidateBoxValue] x(-4) < 0 || x(-4) >= Space->Shape[0](1)
[UNevarokMLSample::ValidateBoxValue] x(-5) < 0 || x(-5) >= Space->Shape[0](1)
```

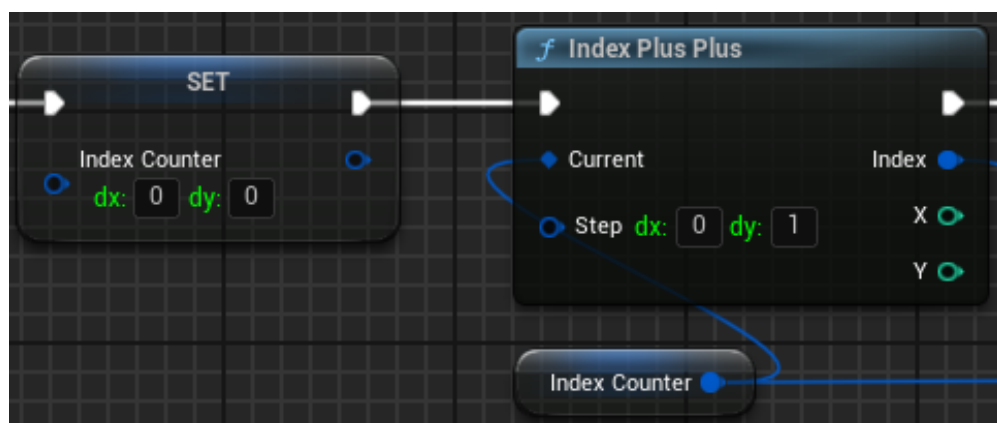
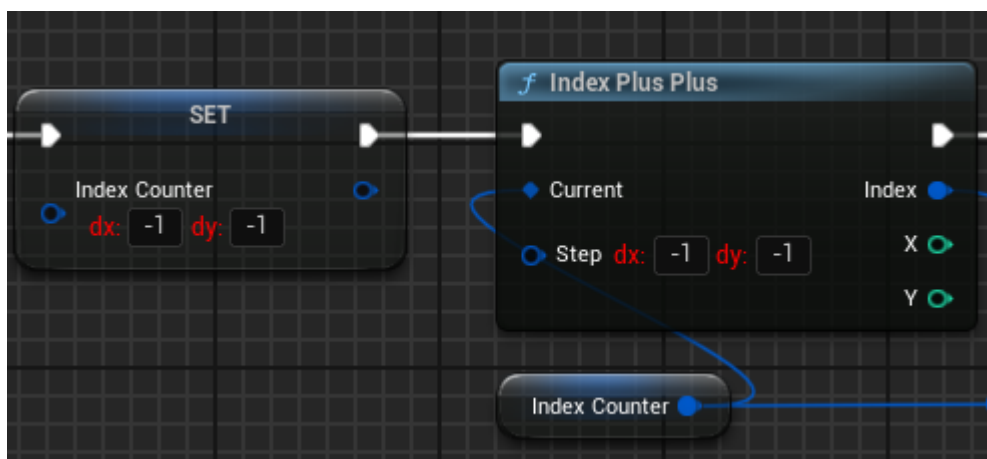
2. Tato chyba je bohužel známým problémem pluginu NevarokML, ale naštěstí má jednoduché řešení.
3. Po otevření projektu klikněte ve spodním levém rohu na tlačítko „Content Drawer“ a přejděte do složky All/Content/NPC/AI, jak je znázorněno na obrázku 3. Zde otevřete „Blueprint“ s názvem NPC_Enviroment.



4. Po otevření „Blueprintu“ byste měli na levé straně vidět sekci „Functions“, kde otevřete funkci „CollectObservations“, jak je znázorněno na obrázku 4. Pokud tuto sekci nevidíte, klikněte nahoře na tlačítko „Window“ a v části „Graph Editor“ zaškrtněte položku „My Blueprint“.



5. Ve funkci „CollectObservations“ se nachází uzel „Set Index Counter“ a pět uzlů s názvem „Index Plus Plus“. Pokud se agent nepohybuje nebo se zobrazila chyba z obrázku 1, je pravděpodobné, že tyto uzly vypadají tak, jak je znázorněno na obrázku 5. V uzlu „Set Index Counter“ je třeba nahradit dvě červené záporné jedničky nulami. Ve všech uzlech „Index Plus Plus“ pak nahraďte záporné jedničky tak, že do prvního políčka zadáte nulu a do druhého jedničku, jak je znázorněno na obrázku 5.



6. Tímto je agent opraven a měl by nyní fungovat správně.